

基于红外干涉物理建模与贝叶斯推断的外延层厚度测定问题研究

摘要

在半导体材料检测领域，碳化硅外延层厚度的精确测量对于器件性能和可靠性具有重要意义。针对红外干涉法测量外延层厚度的数学建模问题，本文系统建立了完整的理论模型与计算算法，并通过实验数据验证了方法的准确性和实用性。

针对问题一，本文构建了基于单次反射透射干涉的外延层厚度数学模型，实现了厚度与反射率的精确对应。模型首先利用**菲涅尔反射**理论和干涉原理推导反射率与厚度的关系式，并结合几何分析和光程差定义得到两束光的光程差表达式。在充分考虑不同材料折射率特性和半波损失影响的基础上，建立了干涉相长条件下光程差与波长的定量关系，进一步通过波长与波数互为倒数的关系，构建了光程差与波数的对应模型，为外延层厚度的精确测定提供了理论依据。

针对问题二，本文在问题一建立的理论模型基础上，引入碳化硅在 $1785\text{--}4000\text{ cm}^{-1}$ 波数区间内的**塞尔迈耶方程**，构建了包含折射率与厚度两个关键参数的干涉模型。为克服直接求解困难，采用相邻极大值点联立消元的方法，并结合实验反射率光谱数据进行厚度计算。通过对 $2000\text{--}4000\text{ cm}^{-1}$ 区间内的干涉极值点进行提取与多次计算，最终得到厚度为 $7.7310\text{ }\mu\text{m}$ 。进一步对比分析表明，不同入射角度下的厚度对称相对差仅为 0.7506% ，低于常见实验测量不确定度范围，验证了所建立模型在不同入射条件下的稳定性与可靠性。

针对问题三，本文深入分析了多光束干涉对外延层厚度测量的影响。通过**快速傅里叶变换**（FFT）和物理条件判定，确认实验样品满足多光束干涉的形成条件。进一步建立了基于**Drude-Lorentz 模型**、偏振效应及**菲涅尔方程**的**多光束干涉物理模型**，严格考虑高阶光束累加、材料色散与吸收、表面粗糙度、厚度与入射角分布及仪器分辨率等非理想效应。在此基础上，结合 FFT 分析、**高斯-埃尔米特积分**方法以及**信赖域反射**算法，设计了对外延层厚度及相关物理参数进行高精度、鲁棒拟合的计算流程。将该模型应用于硅与碳化硅外延层的厚度反演，分别得到 $3.3912\text{ }\mu\text{m}$ 和 $7.0873\text{ }\mu\text{m}$ 的结果，有效消除了多光束干涉引入的偏差，显著提升了拟合精度与测量可靠性。

在确定性最小二乘获得高质量点估计的基础上，本文进一步引入**贝叶斯推断**对模型参数进行不确定度量化与模型比较。构建以多光束干涉物理模型为前向映射、以**Student-t 噪声**为似然的概率模型，结合满足物理约束的先验信息，并采用**仿射不变 MCMC** 实现对单角度与多角度实验数据的分层联合后验推断。该方法不仅获得了外延层厚度及关键材料参数的**后验分布与 95% 可信区间**，还通过**后验预测检查**与**WAIC/LOO** 指标完成了不同物理假设的比较与验证。

综上所述，本文针对硅基外延层厚度测量问题，构建了融合干涉光学建模与统计推断的分析框架。该框架实现了由确定性拟合到贝叶斯推断的完整流程，不仅获得了高精度的厚度反演结果，还对关键参数的不确定性进行了量化，并对不同物理假设进行了系统比较。研究成果为红外光谱测量与材料表征提供了一种可推广、可扩展的分析范式。

关键词：红外干涉法 菲涅尔反射 塞尔迈耶方程 非线性最小二乘拟合 贝叶斯推断

一、问题重述

1.1 问题背景

随着第三代半导体材料在功率电子、射频通信及极端环境应用领域的战略地位日益凸显，其外延层的质量与参数控制成为高性能器件实现的关键瓶颈。SiC 材料以其优异的宽禁带、高击穿电场和高热导率等特性，显著超越传统硅基材料，但这些性能的充分发挥高度依赖于精确控制的外延层厚度。外延层厚度直接影响器件的耐压能力、导通损耗和工作频率，因此，发展一套无损、快速、高精度的外延层厚度测量技术是 SiC 外延工艺研发与质量控制的核心需求^[1]。

传统的破坏性测量方法已难以满足现代半导体制造对效率和成本的严苛要求，促使红外干涉光谱法成为业界标准测量手段之一^[2]。该技术利用红外光在外延层内外界面的反射干涉现象，通过分析反射光谱中的干涉条纹来反演外延层厚度，其非接触性、无损性和快速性使其在工业界和学术界得到广泛应用。

然而，红外干涉法在实际应用中也存在挑战。首先，半导体外延层的折射率受掺杂和波长影响，模型较为复杂，给厚度计算带来不确定性。其次，多光束干涉效应可能在高反射率界面或薄外延层条件下出现，从而改变光谱特征，导致基于简单模型的厚度计算产生偏差。对多光束干涉的准确建模与修正，是保证红外干涉法测量结果可靠性和精度的关键，尤其在先进 SiC 器件制造中更显重要。

1.2 问题提出

随着 SiC 在高端应用中地位提升，外延层厚度精确测量成为关键，但红外干涉法虽具无损高效优势，仍受折射率色散和多光束干涉等因素制约，亟需建立更完善的建模与修正方法以保障测量精度。

基于上述背景，本研究旨在深入探讨红外干涉法在 SiC 和 Si 外延层厚度测量中的挑战与优化策略，具体包括以下三个核心问题：

1) **单次反射干涉模型的建立**：在仅考虑外延层和衬底界面只有一次反射、透射所产生的干涉条纹的情形，建立确定外延层厚度的数学模型。

2) **基于单次反射模型的算法设计与应用**：基于问题 1 的模型，设计算法并用附件 1、附件 2 的 SiC 光谱数据计算外延层厚度，分析结果可靠性。

3) **多光束干涉效应的分析与修正**：分析多光束干涉的产生条件及其对厚度计算的影响。结合附件 3、附件 4 的 Si 光谱数据判断是否存在多光束干涉，建立相应模型和算法并计算结果。检验附件 1 和附件 2 中的 SiC 样品是否存在多光束干涉效应；若有，提出修正方法，建立更精细模型，并给出修正后的精确厚度结果。

二、问题分析

2.1 对于问题一的分析：

对于问题一，核心在于建立外延层厚度与干涉光谱之间的定量联系。根据光学干涉原理，干涉条纹的形成由光程差决定，而光程差又取决于外延层厚度、折射率以及入射角等参数。在仅考虑单次反射的情况下，可以将外延层厚度视为光程差的函数，从而得到厚度与光谱中干涉峰位置之间的数学对应关系。

在实际计算中，可以利用附件 1、2 提供的光谱实测数据，提取反射率峰值对应的波数作为相长干涉的特征点。再结合文献中给出的碳化硅塞尔迈耶方程^[3]，建立厚度与

波数的函数关系。通过算法识别多个干涉峰并进行数值求解，就能够反推出外延层厚度，并进一步分析结果的可靠性。

2.2 对于问题二的分析：

在问题二中，若直接利用问题一中建立的干涉模型求解外延层厚度，计算过程较为复杂。为此，首先查阅相关文献以获得碳化硅在给定波数范围内的塞尔迈耶方程，从而将折射率表示为波数的函数。随后，绘制附件（1）和附件（2）提供的反射率随波数变化的曲线，并利用计算机程序自动检索多组相邻的极大值点。每组极大值点包含两个峰值，对应两组干涉条件，其数学形式与问题一中的模型一致。通过对每组峰值建立方程并联立消元，可求解出未知参数。

由于不同组极大值点可分别给出一个厚度估计值，在计算中对多组结果进行求解并取平均值，以减小实验噪声和数值计算带来的误差，从而获得更为可靠的外延层厚度估计结果。为进一步佐证结果的一致性，可比较两组独立数据得到的平均厚度，并用对称相对差量化它们的一致性。在典型实验误差水平之内的相对差进一步证明了两组数据和处理方法的一致性与可靠性。

2.3 对于问题三的分析：

针对问题三，本文的研究工作主要聚焦于以下三个方面：

1) 硅晶圆片光谱特性分析与外延层厚度反演：

通过对光谱数据进行深入分析，判定其是否存在典型的多光束干涉现象。在此基础上，建立严格的物理与数学模型，并提出相应的数值求解算法，实现对外延层厚度的精确计算。

2) 碳化硅厚度计算的模型修正与算法优化：

将精确模型引入碳化硅体系，对传统双光束干涉模型的非理想效应进行修正。进一步设计并实现一套鲁棒的数值优化方法，用于求解该类复杂的高维非线性模型，从而提升厚度反演结果的准确性与稳定性。

3) 模型的统计推断与不确定性量化：

在确定性拟合的基础上，引入贝叶斯推断方法，对外延层厚度及相关光学参数进行后验分布估计与不确定性量化。通过多角度数据的分层联合建模与 MCMC 采样，不仅提高了参数估计的稳健性，还能够对不同物理假设进行模型比较与合理选择，为复杂材料体系的精确表征提供了新的思路。

三、模型假设与符号说明

3.1 模型基本假设

- 1) 空气折射率取 1，且折射率满足：衬底>外延层>空气。
- 2) 入射的红外光为寻常光。
- 3) 外延层和衬底为 4H-SiC 晶体。
- 4) 界面具有高平整度与平行度。

3.2 符号说明

表1 符号说明

符号	含义	单位
θ_1	红外光入射角	°（度）
n	外延层折射率	无
θ_2	红外光折射角	°（度）
OPD	光程差	cm（厘米）
λ	波长	cm（厘米）
σ	波数	cm ⁻¹
d	外延层厚度	μm
σ_1	相邻极大值中的较小者	μm ⁻¹
σ_2	相邻极大值中的较大者	μm ⁻¹
R	反射率	无
m	干涉级次	无
OPD_0	基本光程差	cm（厘米）
δ	相位差	rad（弧度）

这里只列出论文各部分通用符号，个别模型单独使用的符号在首次引用时会进行说明。

四、模型建立与求解

4.1 问题一模型建立与求解

4.1.1 问题一模型建立

如图 1 所示，由几何对称性可知，光线从点 C 到 F 与从点 D 到 E 的光程相等，因此两束光的光程差主要来源于直线路径 AD 与折线路径 ABC 的差异。

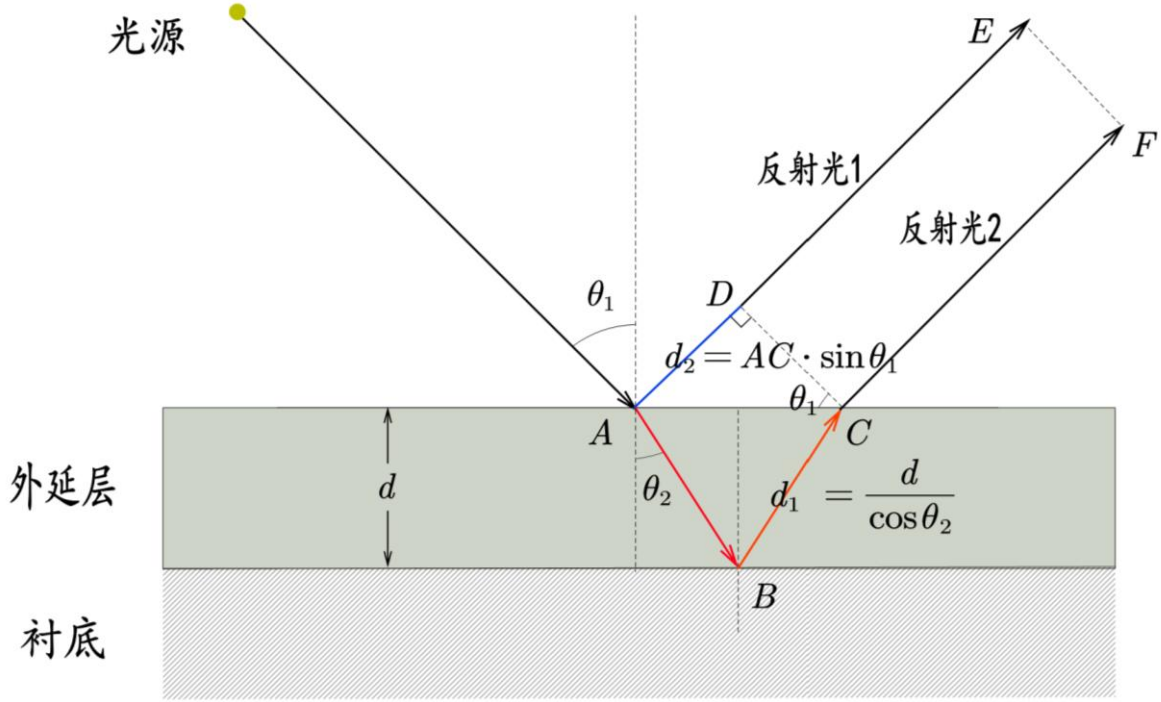


图1 光线在外延层干涉示意图

设入射角为 θ_1 ，外延层折射率为 n ，折射角为 θ_2 ，空气折射率近似取 1。根据斯涅尔定律^[4]，有

$$\sin\theta_1 = n \sin\theta_2 \quad (1)$$

光线在厚度为 d 的外延层中传播的几何长度为：

$$AB + BC = 2d_1 = \frac{2d}{\cos\theta_2} \quad (2)$$

其中 d_1 为 BC 的路径长。与此同时，反射光 1 在空气中相较于反射光 2 多走的距离 d_2 为：

$$AC \cdot \sin(\theta_1) = (2d \cdot \tan(\theta_2)) \cdot \sin(\theta_1) \quad (3)$$

综合上述结果，两束光的光程差为：

$$OPD = 2d_1 - d_2 = \frac{2nd}{\cos\theta_2} - (2d \tan\theta_2) \sin\theta_1 = 2nd \cos(\theta_2) \quad (4)$$

进一步利用斯涅尔定律消去 θ_2 ，可得以入射角 θ_1 表示的光程差为：

$$OPD = 2d\sqrt{n^2 - \sin^2\theta_1} \quad (5)$$

在模型的基本假设下，折射率大小满足衬底>外延层>空气，因此光线在外延层和衬底界面处反射时均发生一次半波损失，两次效应叠加后等效为相长干涉。由此可得反射率极大值的条件为：

$$OPD = m\lambda, \quad m \in \mathbb{Z}^+ \quad (6)$$

由波数定义 $\sigma = \frac{1}{\lambda}$ ，上述条件可进一步表示为：

$$OPD = 2d\sqrt{n(\sigma)^2 - \sin^2\theta_1} = \frac{m}{\sigma}, \quad (7)$$

其中 $n(\sigma)$ 表示折射率随波数的色散关系，在问题二中将给出其具体形式。

4.2 问题二模型建立与求解

4.2.1 问题二模型建立

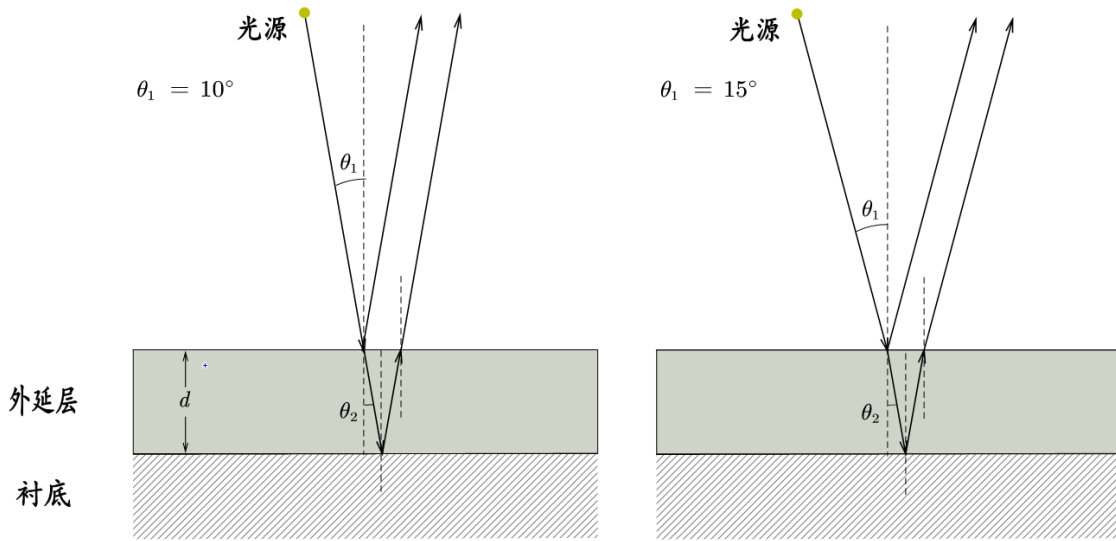


图2 两种入射角度下模型示意图

本文注意到问题一中建立的最终模型中仍含有未知的折射率 $n(\sigma)$ 。查阅资料可知，在波数范围 $1785\text{--}4000\text{ cm}^{-1}$ 内，碳化硅(SiC)的折射率满足塞尔迈耶方程^[5]：

$$n^2(\sigma) = 4.588 + \frac{1.975}{1 - 0.116\sigma^2}, \quad (8)$$

其中 σ 表示波数（单位 μm^{-1} ）。假设入射红外光为寻常光。由干涉条件可得：

$$2d\sqrt{n(\sigma)^2 - \sin^2\theta_1} = \frac{m}{\sigma}, \quad (9)$$

直接由该式求解 d 较为困难，因此考虑相邻的两个极大值点。设其波数分别为 σ_1, σ_2 则：

$$2d\sqrt{n(\sigma_1)^2 - \sin^2\theta_1} = \frac{m}{\sigma_1}, \quad 2d\sqrt{n(\sigma_2)^2 - \sin^2\theta_1} = \frac{m+1}{\sigma_2} \quad (10)$$

将方程(1)、(2)联立消去 m ，并代入塞尔迈耶方程，可得外延层厚度 d 的解析表达式：

$$d = \frac{1}{2\left(\sigma_2\sqrt{4.588 + \frac{1.975}{1 - 0.116\sigma_2^2} - \sin^2\theta_1} - \sigma_1\sqrt{4.588 + \frac{1.975}{1 - 0.116\sigma_1^2} - \sin^2\theta_1}\right)}. \quad (11)$$

4.2.2 问题二模型求解

根据 4.2.1 节中对 SiC 外延层干涉模型的推导，外延层厚度 d 的求解思路如下：

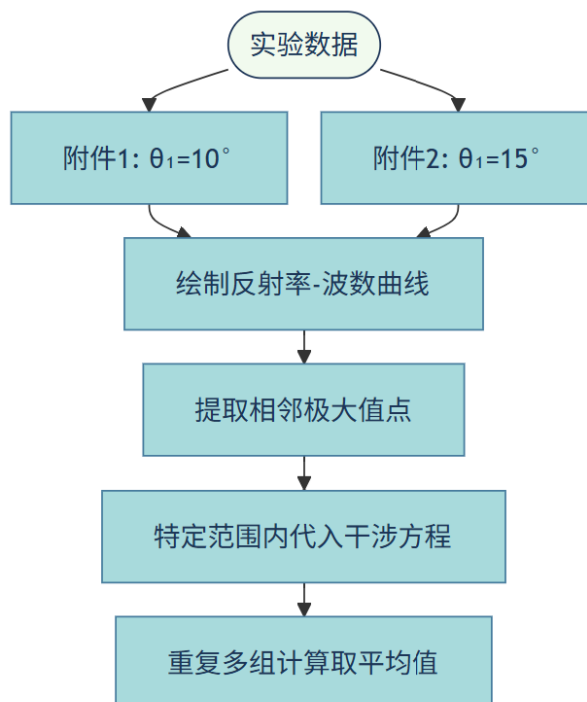


图3 问题二模型求解流程图

Step 1: 数据预处理

在外延层厚度 d 的求解过程中，首先需要对实验数据进行预处理。根据附件 1 ($\theta_1 = 10^\circ$)与附件 2 ($\theta_1 = 15^\circ$)所给出的反射率数据，绘制下图 4 反射率-波数曲线，从而直观地反映出干涉条纹的分布情况。曲线中的极大值点对应于干涉增强的位置，为后续厚度计算提供了基础。

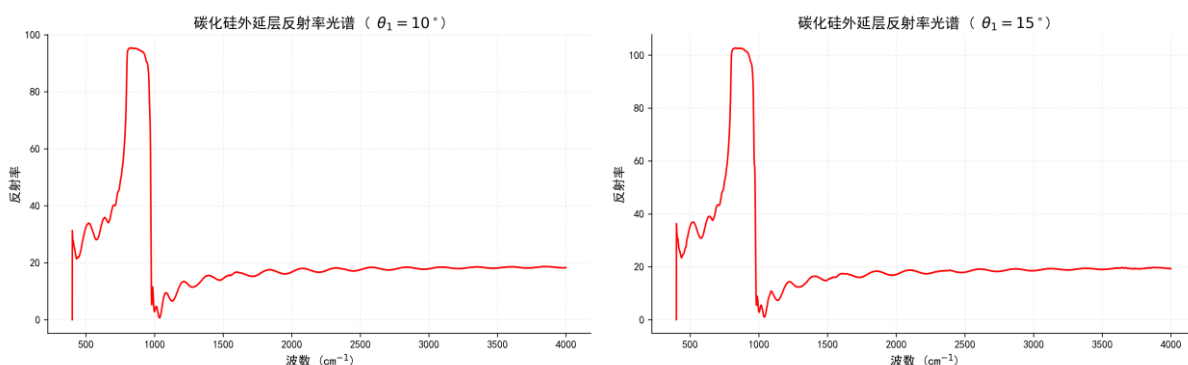


图4 碳化硅外延层反射率光谱

Step 2: 寻找多组相邻极大值点

其次，需要从实验曲线中提取合适的相邻极大值点。由于碳化硅的塞尔迈耶方程仅在波数区间 $1785\text{--}4000\text{ cm}^{-1}$ 内有效，因此厚度的计算应限制在该区间内。

1) $\theta_1 = 10^\circ$, 共检测到 15 个波峰，如图 5 所示。

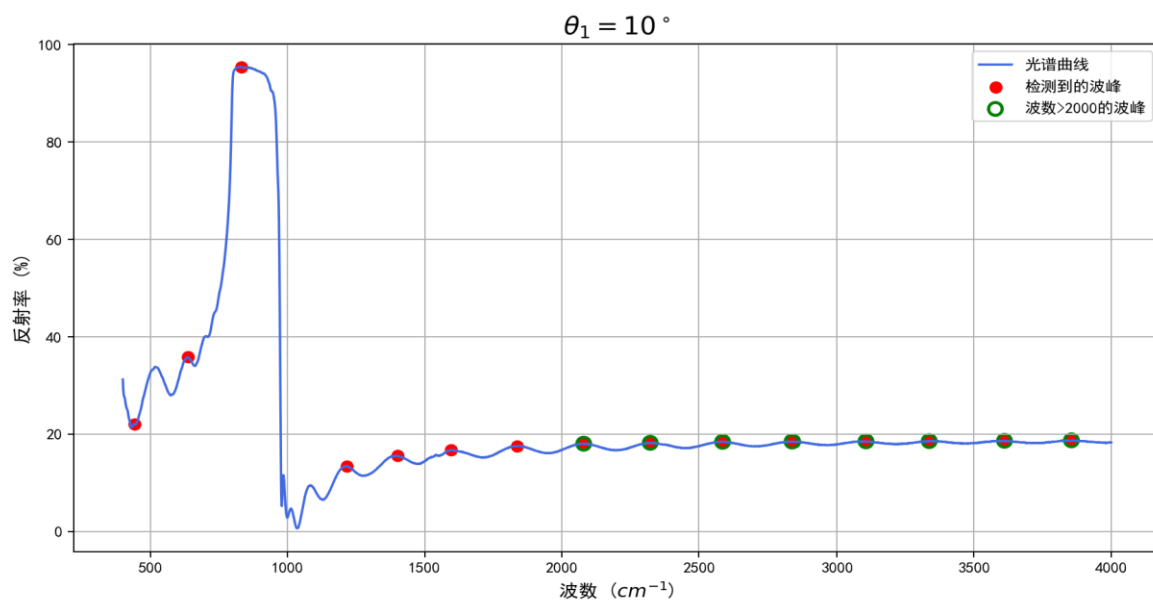


图5 $\theta_1 = 10^\circ$ 时碳化硅外延层反射率曲线及检测波峰

各波峰的位置及相邻波数差值列于表 2。

表2 $\theta_1 = 10^\circ$ 时各波峰的位置及相邻波数差值

波峰编号	波峰值	差值
波峰 1	442.10	-
波峰 2	636.39	194.29
波峰 3	832.62	196.22
波峰 4	1216.38	383.76
波峰 5	1401.51	185.13
波峰 6	1595.32	193.81
波峰 7	1836.87	241.54
波峰 8	2079.85	242.99
波峰 9	2322.36	242.51
波峰 10	2586.07	263.72
波峰 11	2839.18	253.11
波峰 12	3106.28	267.09
波峰 13	3336.73	230.45
波峰 14	3611.05	274.33
波峰 15	3855.01	243.95

2) $\theta_2 = 15^\circ$, 共检测到 14 个波峰, 如图 6 所示。

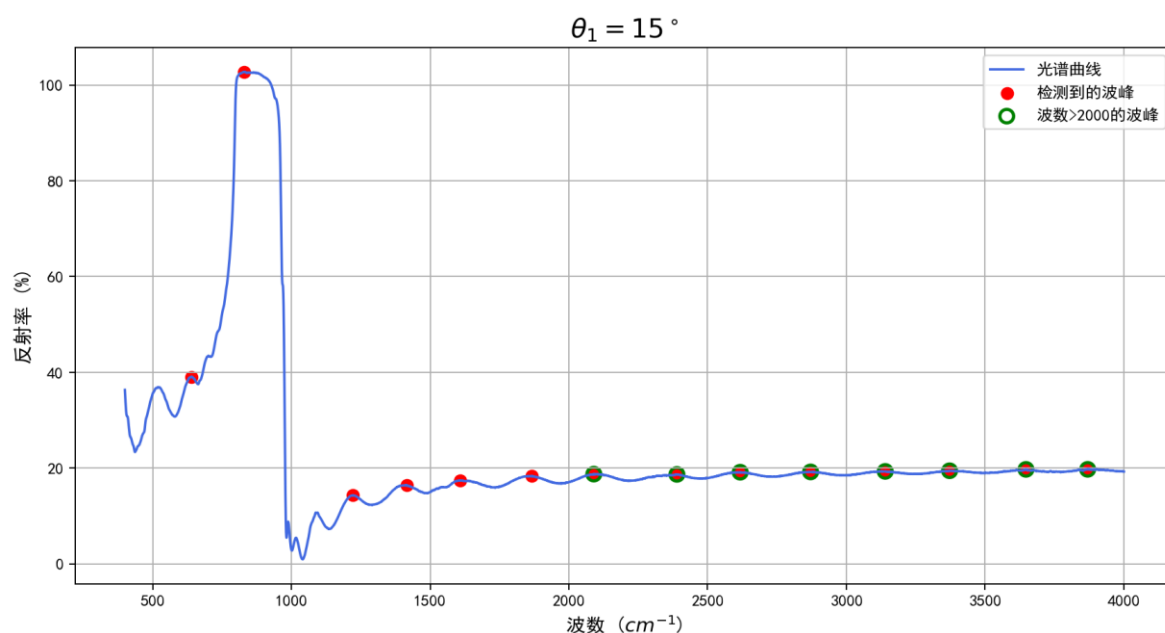


图6 $\theta_1 = 15^\circ$ 时碳化硅外延层反射率曲线及检测波峰

各波峰的位置及相邻波数差值列于表 3。

表3 $\theta_1 = 15^\circ$ 时各波峰的位置及相邻波数差值

波峰编号	波峰值	差值
波峰 1	639.77	-
波峰 2	829.72	189.95
波峰 3	1221.68	391.96
波峰 4	1416.46	194.78
波峰 5	1607.38	190.92
波峰 6	1866.76	259.38
波峰 7	2089.98	223.22
波峰 8	2389.37	299.39
波峰 9	2617.41	228.04
波峰 10	2871.01	253.59
波峰 11	3140.03	269.02
波峰 12	3371.92	231.90
波峰 13	3647.21	275.29
波峰 14	3869.47	222.26

Step 3: 代入方程求解

进一步地观察发现，当波数大于 2000 cm^{-1} 时，反射率曲线逐渐趋近于正弦形式，干涉极值点的分布更加均匀、规律性更强，同时在波数 1000 cm^{-1} 附近，SiC 光谱出现一个陡峭的反射峰，这是一个由材料本身内在物理性质决定的固态物理学特征，被称为“剩余射线带”。

通过查阅相关文献可知，其形成机制可从晶格微观结构与光-物质相互作用的角度理解^[6]：碳化硅为高度有序的晶体，红外光作为电磁波，其能量可与晶格的声子模式相匹配。当入射红外光的频率恰好与 SiC 晶格中某一特定振动模式（通常为横向光学声子，TO-Phonon）的本征频率相同时，光与晶格发生强烈共振。此时，光能几乎完全被晶格吸收，无法有效进入材料内部，导致大部分能量在表面被反射，从而在反射光谱中呈现出显著的反射峰^[7]。

基于这一物理特征，为避免受强吸收峰影响，本文在实际操作中拟合波段选定为 $2000\text{--}4000\text{ cm}^{-1}$ 范围内的相邻极大值点，以确保对薄膜光学性质的精确反演。

极值点的波数经过单位换算后，分别记为 σ_1, σ_2 （单位为 μm^{-1} ），具体数值见表 4。将这些极大值点代入由干涉条件推导得到的厚度公式，即可计算得到对应的外延层厚度。

Step 4: 多次测量取平均值

最后，为了降低偶然误差的影响，需要对多组不同的相邻极大值点进行重复计算，得到多组厚度数据，并取其平均值作为最终的外延层厚度估计值。通过这种方法，不仅可以有效利用实验曲线中的干涉信息，而且能够提高厚度测量的稳定性与可靠性。

表4 2000–4000 cm^{-1} 范围内时相邻波峰及厚度计算结果

$\theta_1 = 10^\circ$			$\theta_1 = 15^\circ$		
$\sigma_1(\mu\text{m}^{-1})$	$\sigma_2(\mu\text{m}^{-1})$	厚度 $d(\mu\text{m})$	$\sigma_1(\mu\text{m}^{-1})$	$\sigma_2(\mu\text{m}^{-1})$	厚度 $d(\mu\text{m})$
0.2080	0.2322	8.04603	0.2090	0.2389	6.53492
0.2322	0.2586	7.39416	0.2389	0.2617	8.57391
0.2586	0.2839	7.6985	0.2617	0.2871	7.70471
0.2839	0.3106	7.28968	0.2871	0.3140	7.25696
0.3106	0.3337	8.4417	0.3140	0.3372	8.41149
0.3337	0.3611	7.08506	0.3372	0.3647	7.07901
0.3611	0.3855	7.95907	0.3647	0.3869	8.75943
平均厚度	-	7.70203	平均厚度	-	7.76006

4.2.3 问题二结果分析

- 入射角为 10° 时，根据附件 1 所给实验数据计算得到 $\bar{d}_{10^\circ}$ 的平均值为 **7.70203 μm** 。
- 入射角为 15° 时，根据附件 2 对应数据计算得到 $\bar{d}_{15^\circ}$ 平均值为 **7.76006 μm** 。

由上述两组结果取算术平均，可得 $d = \mathbf{7.7310\text{ }\mu\text{m}}$ 。为了定量评估两组数据的一致性，采用对称相对差：

$$\frac{|\bar{d}_{15^\circ} - \bar{d}_{10^\circ}|}{(\bar{d}_{10^\circ} + \bar{d}_{15^\circ})/2} \times 100\% \quad (12)$$

代入数据计算得二者的相对相差为 **0.7506%**，考虑到实验测量误差通常在 $\pm 1\%$ 左右，该相对差小于测量不确定度范围，表明两组数据高度一致。由此可认为所建立的光学模型在不同入射角下具有良好的可靠性。

4.3 问题三模型建立与求解

在 4.1 与 4.2 节中，本文讨论了光波在单个界面上的一次反射情形。然而在实际结构中，光波不仅会在外延层表面发生反射，还会在外延层与衬底界面多次反射与透射。不同往返路径的光束相干叠加后，便会产生典型的多光束干涉现象（见图 7）。因此，有必要进一步推导多光束干涉形成的必要条件以及分析其造成的影响。

4.3.1 产生多光束干涉的必要条件推导

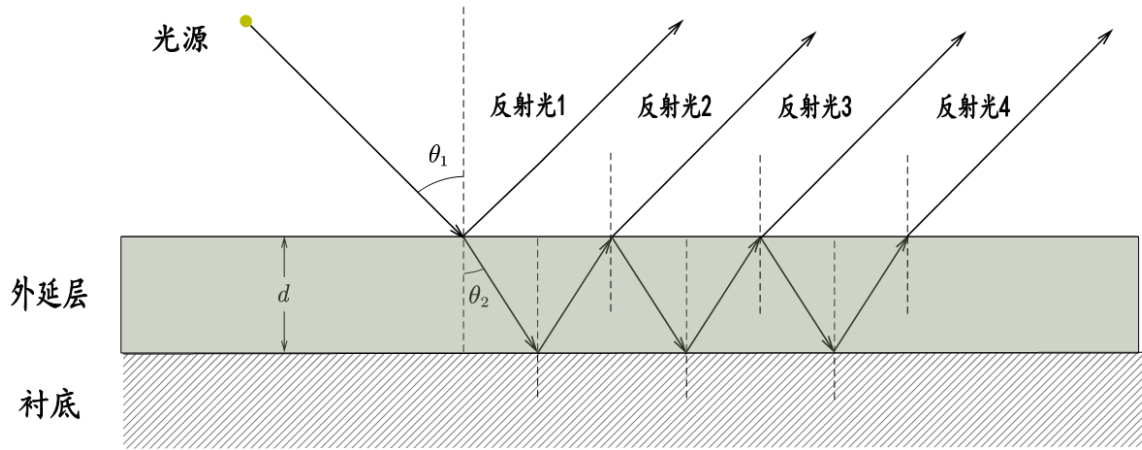


图7 多光束干涉的示意图

首先，从实验数据出发对光谱进行快速傅里叶变换（FFT），可得到光谱在光程差（OPD）域的频谱分布。若干涉仅由单对相干光产生，则其强度分布可表示为：

$$R(\sigma) = A_0 + A_1 \cos(\delta), \quad \delta = 2\pi\sigma \text{ OPD}_0, \quad \text{OPD}_0 = 2nd \cos\theta_2 \quad (13)$$

其中 $R(\sigma)$ 表示光强关于波数 σ 的分布； A_0 为背景光强， A_1 为干涉调制振幅，二者与光源强度及系统耦合效率等有关； OPD_0 为单程光程差。其 FFT 结果只会在基频 OPD 处出现单一峰。

然而，对于薄膜中存在的多次往返反射，场振幅可写为几何级数：

$$R(\sigma) = A_0 + \sum_{m=1}^{\infty} A_m \cos(2\pi m \sigma \text{ OPD}_0) \quad (14)$$

其中 A_m 表示第 m 次往返反射的调制振幅。因此在 $\sigma \rightarrow \text{OPD}$ 的 FFT 中，必然在 $\text{OPD} = m \cdot \text{OPD}_0 (m=1, 2, 3, \dots)$ 处出现一系列峰。

由此可见：**FFT 中若出现基频及其高次谐波峰列，即为多光束干涉的决定性判据。**

在确立了判据后，本文进一步推导出保证多光束干涉得以形成的物理条件：

1) 界面具有高反射率：

设光波在空气 - 外延层界面的反射率为 R_1 、透射率为 T_1 ，在外延层 - 衬底界面的反射率为 R_2 。光束 1 的强度与 R_1 成正比。光束 2 在穿过顶面、底面反射后再出射，其强度与 $I_2 \propto T_1 R_2 T_1 = T_1^2 R_2$ 成正比。光束 3 为在层内来回反射一次后出射，其强度为

$I_3 \propto T_1^2 R_1 R_2^2$. 可以看出, 高阶光束的强度随 R_1 和 R_2 的乘积呈指数级衰减。因此, 当界面反射率 R_1 或 R_2 较低时, 高阶光束的贡献迅速减小, 系统的干涉行为将退化为典型的双光束干涉。

2) 薄膜材料具有较低吸收率:

在实际材料中, 光在介质中传播会因吸收而衰减。光单次穿过(垂直方向)的透过率为 T_{film} 。光束 2 在层内往返一次, 其强度需额外乘以 T_{film}^2 , 光束 3 则乘以 T_{film}^4 , 以此类推。若材料对所考虑波段的光吸收较强(即 $T_{\text{film}} \ll 1$), 高阶光束在多次内部反射后能量将几乎完全耗散, 其贡献可忽略不计。

3) 界面具有高平整度与平行度:

多光束干涉要求光在界面上发生接近理想的镜面反射, 即反射角等于入射角, 且所有出射光束保持平行。如果界面粗糙, 光将发生漫反射, 能量分布在各个方向, 难以形成方向一致的平行光束。此外, 若顶面与底面不严格平行, 高阶光束的出射方向将发生偏离, 干涉条纹的可观测性随之降低。因此, 为产生稳定的多光束干涉, 两个界面必须具有光学级别的平滑度且严格平行。

4.3.2 多光束干涉对外延层厚度计算精度产生影响的分析

高阶光束的相干叠加会改变干涉峰的形状与振幅, 从而对厚度反演产生显著影响。

1) 多光束干涉改变干涉条纹的形状:

在双光束近似下, 干涉条纹呈平滑的余弦或正弦函数, 峰谷圆润且对称。而多光束干涉中, 多束同相光的能量叠加会使相长干涉峰变得尖锐、陡峭, 相消干涉区域则变宽、平坦。这种尖峰和平坦谷的形成使光谱局部结构更复杂, 增加厚度反演难度。

2) 峰值定位的鲁棒性下降:

尖锐峰对噪声极其敏感, 峰顶位置的微小扰动即可导致波数的漂移。若峰形存在非对称性, 算法可能系统性偏向一侧, 从而引入系统误差。这些随机误差和系统误差在厚度计算中会通过相邻峰的数据累积传递, 进一步降低反演精度。

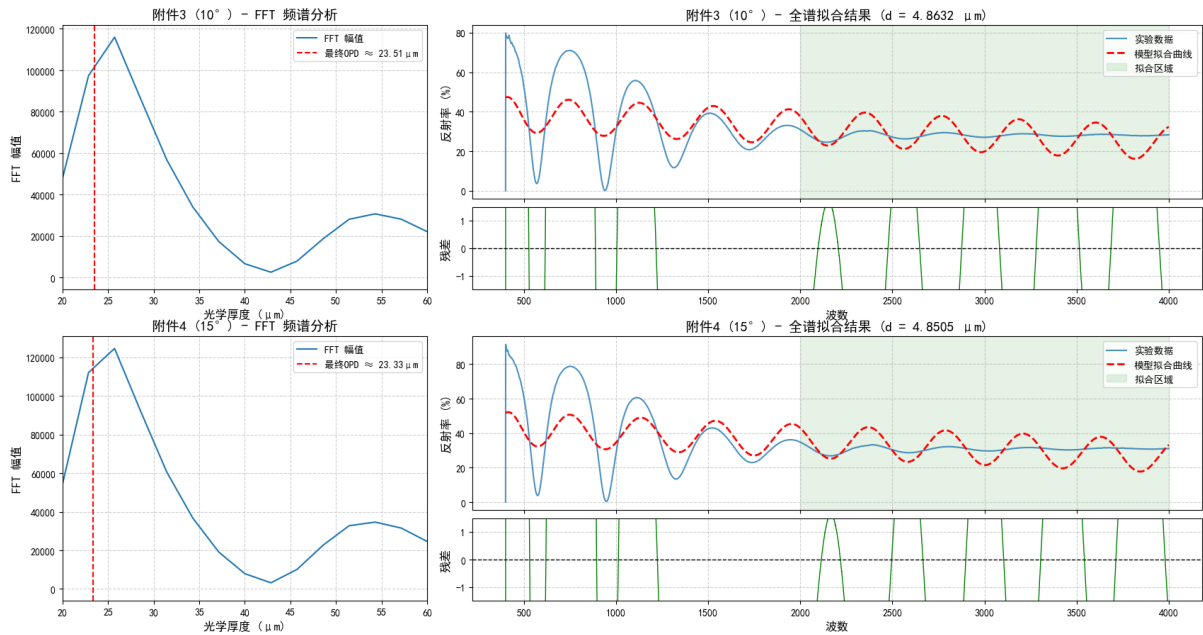


图8 Si 双光束模型拟合 Si

如图 8 所示，若忽略多光束干涉，仅采用双光束近似模型，不仅会导致峰位偏移和振幅误差，还会使拟合曲线无法准确复现实验光谱，厚度计算精度明显下降，尤其在高折射率或厚层样品中更为显著。

4.3.3 问题三模型建立

Step 1: 物理现象的观测与判断

下图 9 展示了对附件(3)、(4)所含光谱数据进行快速傅里叶变换 (FFT) 后得到的结果。该变换将“随波数变化的振荡频率”投影到“光程差 OPD 轴”上，从而揭示了光谱中干涉成分的本征特征。

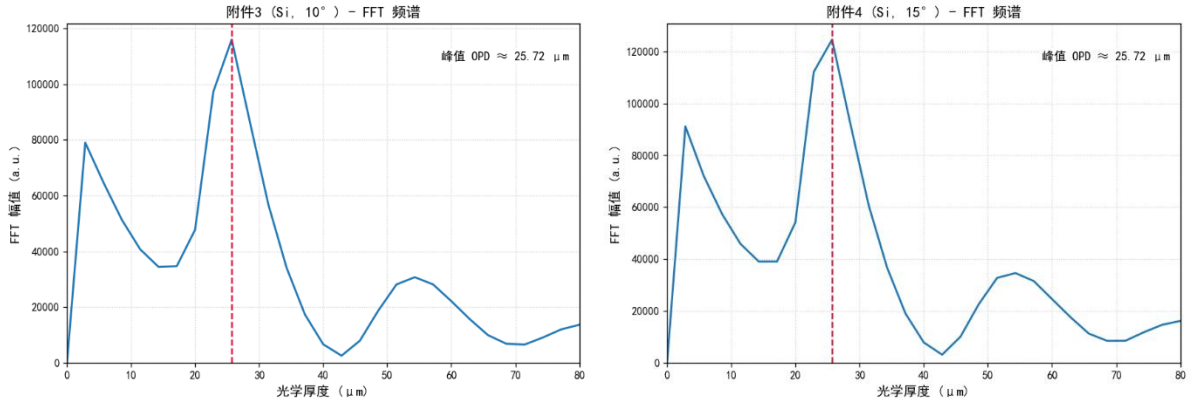


图9 Si 的 FFT 频谱分析

根据 4.3.2 的推导结果，附件(3)、(4)的 FFT 频谱中均呈现出基频及等间隔谐波峰列，可明确判定其光谱干涉现象属于典型的多光束干涉。进一步结合实验条件与模型基本假设，可以确认样品同时满足多光束干涉的必要条件：界面反射率较高、材料在考察波段吸收较弱，且晶圆表面加工精度达到光学级别的平整与平行。因此，附件(3)与附件(4)完全符合多光束干涉的产生条件。

Step 2: 物理模型的构建

本文建立的高级模型基于严格的电磁理论，将反射光谱视为光与多层膜系相互作用的宏观体现。理论反射率 R_{model} 的计算包括以下几个核心环节：

1) 材料光学常数的建模：Drude-Lorentz 模型

材料的复介电函数：

$$\varepsilon(\sigma) = \varepsilon_1(\sigma) + i\varepsilon_2(\sigma) \quad (15)$$

是决定光学响应的核心物理量。本文采用经典的 Drude-Lorentz 模型：

$$\varepsilon(\sigma) = \varepsilon_{\infty} - \frac{\nu_p^2}{\sigma^2 + i\gamma_d\sigma} + \sum_{k=1}^N \frac{S_k \nu_{0k}^2}{\nu_{0k}^2 - \sigma^2 - i\gamma_k\sigma} \quad (16)$$

其中：a) ε_1 反映材料的色散效应（折射）； ε_2 反映材料的吸收效应（损耗）；

b) ε_{∞} 为高频介电常数；

c) Drude 项：描述自由载流子吸收，由等离子体频率 ν_p 与阻尼系数 γ_d 控制；

d) Lorentz 项：描述晶格振动的共振吸收，由振子强度 S_k 、中心频率 ν_{0k} 及阻尼系数 γ_k 决定。本文中设置两个 Lorentz 振子 $N = 2$ ，以覆盖可能存在的多个声子吸收峰。由复介电函数可直接得到复折射率：

$$N(\sigma) = n(\sigma) + ik(\sigma) \quad (17)$$

从而自然包含了色散（ n 随 σ 变化）与吸收（ $k > 0$ ）效应。

2) 多层膜反射率的计算：考虑偏振与菲涅尔方程

考虑到实际红外光源并非理想单一偏振光，本文分别计算 s 偏振与 p 偏振在“空气-外延层-衬底”三层结构中的反射系数，并按比例加权：

$$r_{s,p} = \frac{r_{01,s,p} + r_{12,s,p} e^{2i\beta}}{1 + r_{01,s,p} r_{12,s,p} e^{2i\beta}}, \quad \beta = 2\pi\sigma d N \cos\theta_1 \quad (18)$$

其中 $r_{01,s,p}$ 与 $r_{12,s,p}$ 分别为界面空气-外延层与外延层-衬底的菲涅耳反射系数。

最终反射率为：

$$R = (1 - p_{\text{frac}})|r_s|^2 + p_{\text{frac}}|r_p|^2 \quad (19)$$

其中 p_{frac} 表示 p 偏振分量比例，是模型的待定参数之一。

3) 非理想效应的集成

为使模型更接近实验实况，引入以下关键修正：

- 表面粗糙度(σ_{nm})：通过 Debye--Waller 因子修正菲涅尔系数，从而考虑界面不理想带来的散射与相位扰动。该效应在高波数区域尤为显著。
- 厚度与角度不均匀性(σ_d, σ_{ang})：样品厚度或入射光束角度可能存在微小的空间分布。通过高斯-厄米特积分，对厚度和角度在一个高斯分布上进行数值平均，以模拟这种非相干效应。
- 仪器分辨率($FWHM_{inst}$)：任何光谱仪的都有有限的分辨率。将计算出的理想光谱与一个高斯函数进行卷积，以模拟仪器展宽效应对光谱细节的平滑作用。
- 仪器响应函数：探测器的响应可能不是完美的线性。使用线性函数

$$y = \text{offset} + \text{slope} \cdot \sigma + \text{scale} \cdot R_{\text{model}} \quad (20)$$

来校准最终的理论光谱，使其与仪器的真实读数匹配。

综上所述，本模型不仅严格遵循电磁理论，而且系统性地考虑了实验条件下的非理想因素，为后续厚度反演提供了高度精确且物理一致的理论基础。

Step 3: 多物理效应的高参数非线性光学模型的建立

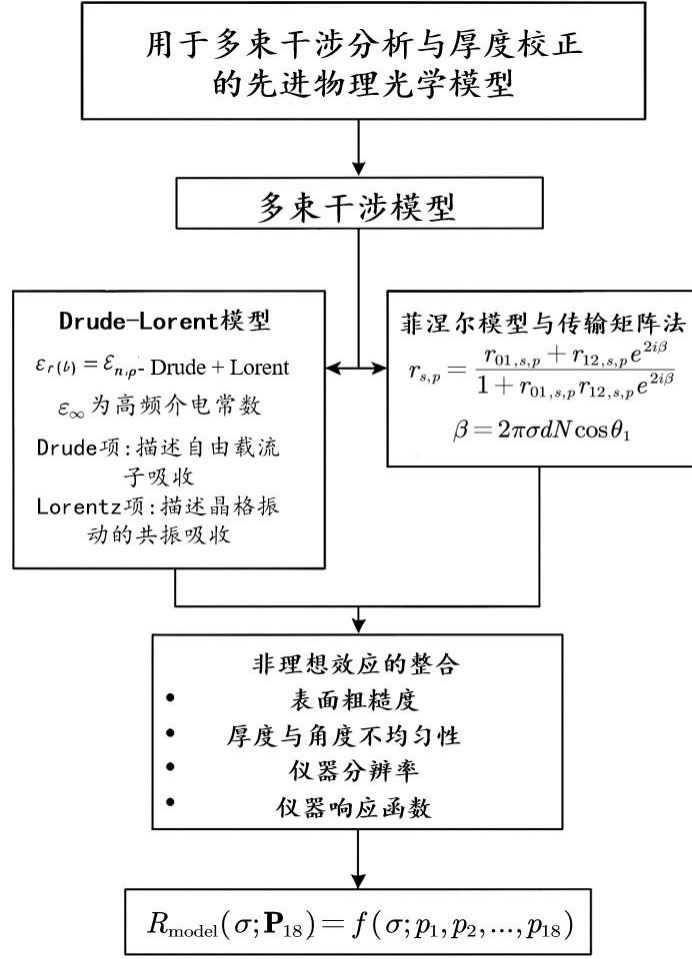


图10 问题三模型搭建流程图

如上图 10 所示, 为了充分考虑多光束干涉、材料色散与吸收、表面粗糙度及仪器分辨率等多种物理效应, 本文建立了一个包含 18 个待求参数的高度非线性模型。

目标函数:

$$\min_{\mathbf{P}_{18}} \sum_i [R_{\text{exp}}(\sigma_i) - R_{\text{model}}(\sigma_i; \mathbf{P}_{18})]^2 \quad (21)$$

模型参数的求解依然以最小化残差平方和 (SSR) 为目标, 求解步骤如下:

1) 初始值估计:

由于原始光谱数据在 500 cm^{-1} 以下存在极端值: 小于 500 cm^{-1} 的波段波峰过于尖锐, 其中 399 cm^{-1} 对应的入射角反射率为 0, 而 400 cm^{-1} 即跃升至三十几, 这部分数据变化过于剧烈, 容易对拟合结果产生不利影响。为了提高拟合的稳定性与可靠性, 本文仅截取光谱数据在 $500\text{--}4000 \text{ cm}^{-1}$ 区间进行分析。通过对截取后的光谱数据进行快速傅里叶变换 (FFT) 分析, 可获得光学厚度 OPD 的稳健初始值, 并由此计算厚度的初始猜测。

2) 高精度拟合:

为了真实地模拟薄膜厚度的微观不均匀性以及入射光束的角度弥散, 本文将厚度 d 和入射角 θ_1 建模为高斯分布。由此产生的对反射率模型的积分, 通过高斯-埃尔米特求积法进行了高效且精确的数值计算。

在非线形最小二乘拟合中, 考虑到模型参数具有明确的物理边界, 本文选用信赖域反射 (TRF) 算法。与无约束的列文伯格-马夸特算法 (LM) 相比, TRF 不仅能够处理带有边

界约束的多参数问题，从而保证能保证优化过程始终在物理有意义的参数空间内进，还在存在噪声和高维情形下表现出更好的鲁棒性与收敛稳定性，更适合本研究的物理参数反演需求。

同时本文结合多起点随机扰动策略，即从多个略微不同的初始参数点启动优化，选取最终 SSR 最小的结果，从而最大程度避免局部最优。

3) 鲁棒性处理：

为了降低实验数据中潜在的异常值和非高斯噪声对拟合结果的影响，本文采用了鲁棒的 soft-L1 损失函数，而非传统的普通平方损失函数。soft-L1 损失对大的残差给予较小的权重，使得拟合结果对少数坏点不敏感，从而增强了参数估计的稳健性。

4.3.4 问题三模型的求解

将上述模型应用于附件 3 与附件 4 的硅光谱数据后，拟合结果表现出极高的精度。

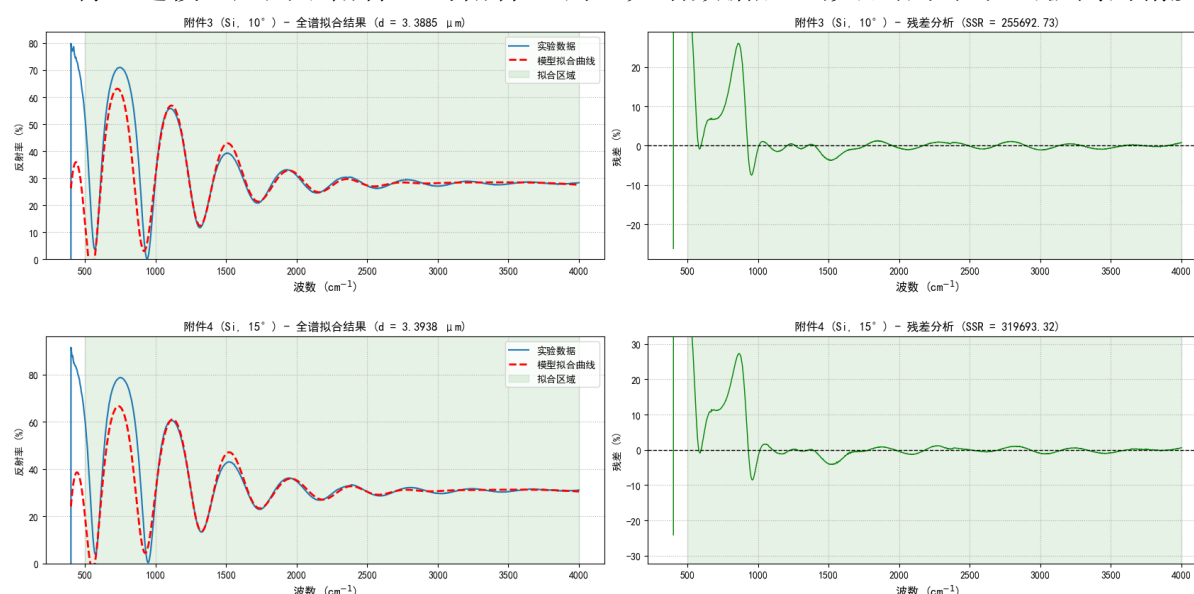


图11 Si 多光束模型拟合

具体地，对不同入射角条件下反演得到的 Si 外延层厚度进行对比，结果如下：

表5 Si 计算厚度

数据集	计算厚度(μm)
附件 3 (Si, 10°)	3.3885
附件 4 (Si, 15°)	3.3938
平均值	3.3912
对称相对差	0.1562%

拟合曲线能够准确复现实验数据的所有细节，包括尖锐的峰形和振幅衰减特征。残差图显示残差值较小且无明显规律。

具体物理参数计算结果如下表：

表6 Si 红外光谱拟合参数数值

符号	含义	单位	附件 3(10°)	附件 4 (15°)
d	厚度	μm	3.3885	3.3938
ε_{∞}	高频介电常数	-	8.3843	8.5451
ν_p	Drude 等离子体频率	cm^{-1}	1163.5765	1178.2599
γ_d	Drude 阻尼	cm^{-1}	1.0000	1.0000
S_1	Lorentz 1 强度	-	0.2469	0.3412
ν_{01}	Lorentz 1 中心	cm^{-1}	1863.7144	1958.2264
γ_1	Lorentz 1 宽度	cm^{-1}	924.1430	1033.9920
S_2	Lorentz 2 强度	-	3.1602	2.7359
ν_{02}	Lorentz 2 中心	cm^{-1}	4420.3489	4363.4355
γ_2	Lorentz 2 宽度	cm^{-1}	2211.8102	1942.6369
p_{frac}	p-偏振光比例	-	0.0000	0.0000
σ_{nm}	表面粗糙度	nm	50.0000	50.0000
σ_d	厚度分布标准差	μm	0.1316	0.1214
σ_{ang}	角度分布标准差	°	2.0000	2.0000
$\text{FWHM}_{\text{inst}}$	仪器分辨率	cm^{-1}	0.1352	0.1799
scale	缩放因子	-	1.2000	1.2000
offset	偏移量	%	-6.7948	-4.4548
slope	斜率	$\%/\text{cm}^{-1}$	-0.0010	-0.0010

最终 SSR 分别为附件 3 (10°) 的 255692.73 和附件 4 (15°) 的 319693.32。该模型及其拟合参数在本文所考察的方案中具有最低的 SSR，表明其对 SiC 光谱的解释能力最强。考虑到有效数据点数目 7259，基于 SSR 和数据点数量计算得到的均方根误差(RMSE)分别为：

$$\text{RMSE}_{\text{附件3}} = \sqrt{\frac{255692.73}{7259}} \approx 5.9350\% \quad \text{RMSE}_{\text{附件4}} = \sqrt{\frac{319693.32}{7259}} \approx 6.6363\% \quad (22)$$

拟合结果表明，多光束物理模型在不同入射角下均能稳定复现硅样品红外光谱。通过考虑厚度与入射角的高斯分布、表面粗糙度及仪器分辨率，模型准确刻画尖锐波峰并捕捉振幅衰减与相位干涉，实现光学厚度与介电性质的高精度反演。残差分布随机且幅值较小，显示模型对实验噪声和非理想效应具有良好鲁棒性。结合 FFT 初始估计与多起点优化，拟合有效避免局部最优，提高参数估计可靠性。

综上，本研究的高级物理光学模型与数值拟合方法为硅薄膜红外光谱的精确表征提供了可靠工具，并可推广至类似材料的厚度反演与介电参数分析。

4.3.5 碳化硅厚度的修正

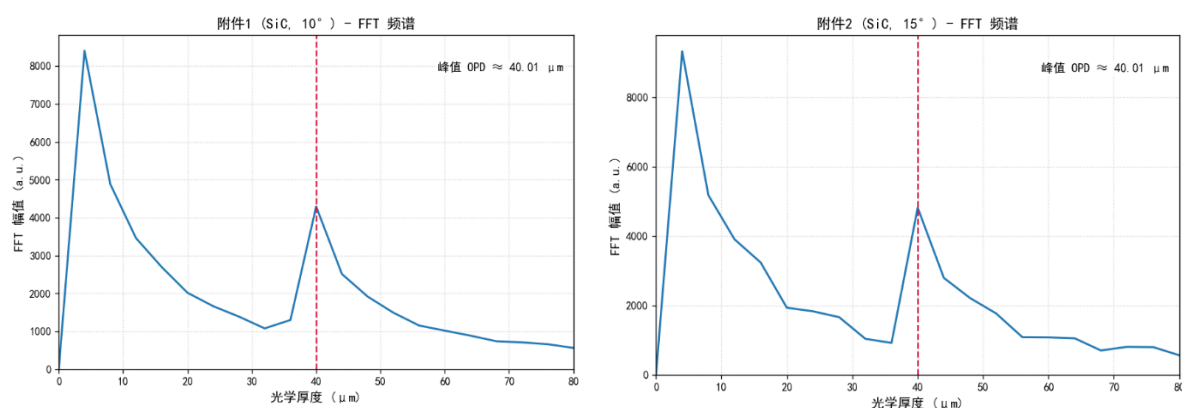


图12 SiC 的 FFT 频谱

如上图 12 所示，附件(1)、(2)的 FFT 频谱显示出基频及等间隔谐波峰列，同时其界面反射率较高、材料吸收较低，且晶圆表面平整平行，因此满足多光束干涉的必要条件。

将上述模型应用于附件 1 与附件 2 的硅光谱数据后，拟合结果表现出极高的精度。

问题三：多光束干涉模型拟合与分析

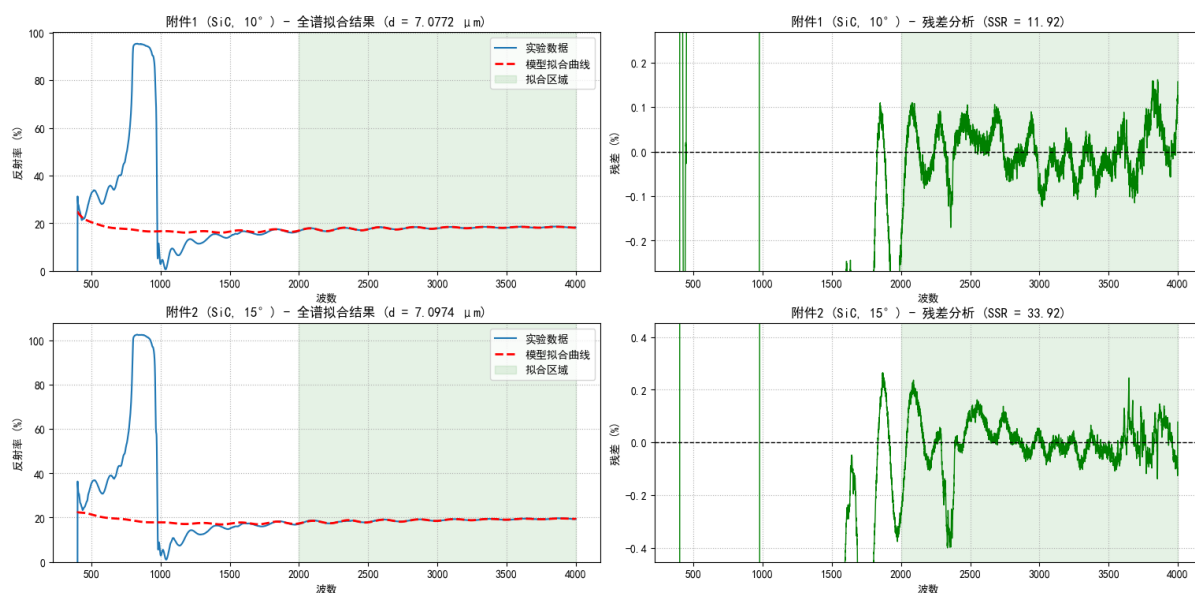


图13 SiC 多光束模型拟合

在此基础上，结合模型的具体计算，本文对不同入射角条件下反演得到的 SiC 外延层厚度进行了系统比较，分析其随入射角变化的规律性与一致性，结果如下：

表7 修正后的 SiC 计算厚度

数据集	计算厚度(μm)
附件 1 (SiC, 10°)	7.0772
附件 2 (SiC, 15°)	7.0974
平均值	7.0873
对称相对差	0.2851%

具体物理参数计算结果如下：

表8 SiC 红外光谱拟合参数数值

符号	中文名称	单位	附件 1 (10°)	附件 2 (15°)
d	厚度	μm	7.0772	7.0974
ε_{∞}	高频介电常数	-	7.29071	6.88351
ν_p	Drude 等离子体频率	cm^{-1}	1199.3	994.1
γ_d	Drude 阻尼	cm^{-1}	1.0	1.0
S_1	Lorentz 1 强度	-	10	10
ν_{01}	Lorentz 1 中心	cm^{-1}	721.1	719.8
γ_1	Lorentz 1 宽度	cm^{-1}	2000.0	2000.0
S_2	Lorentz 2 强度	-	0.204493	0.340097
ν_{02}	Lorentz 2 中心	cm^{-1}	4431.6	5525.6
γ_2	Lorentz 2 宽度	cm^{-1}	3000.0	3000.0
p_{frac}	p-偏振光比例	-	1.000	1.000
σ_{nm}	表面粗糙度	nm	0.70	0.01
σ_d	厚度分布标准差	μm	0.2000	0.2000
σ_{ang}	角度分布标准差	°	2.000	2.000
$\text{FWHM}_{\text{inst}}$	仪器分辨率	cm^{-1}	0.32	0.38
scale	缩放因子	-	0.850	0.850
offset	偏移量	%	0.846	2.020
slope	斜率	$\%/\text{cm}^{-1}$	1.15e-04	2.69e-04

与问题二所建立的简单模型拟合结果相比（附件 1：RMSE = 18.04%；附件 2：RMSE = 22.11%，具体拟合过程详见附录 C-2 代码），本模型的拟合精度明显提高，其中附件 1 的 RMSE 为 5.360%，附件 2 为 9.042%。同时简单模型的对称相对差为 0.7506%，而考虑多光束干涉的本模型对称相对差降至 0.2851%，显示其在拟合附件 1 与附件 2 光谱时的精度更高，对 SiC 光谱的解释能力更强。

最终修正后的 SiC 外延层厚度为 **7.0873 μm** 。该结果在综合考虑多光束干涉、材料色散与吸收、表面粗糙度及仪器分辨率等多种物理效应后得到，代表本研究可提供的最精确、最可靠的厚度测量结果。

五、模型检验

5.1 检验模型构建与推断方法

完成最小二乘拟合并获得外延层厚度点估计后，本文进一步引入贝叶斯推断（Bayesian Inference）框架，以量化参数不确定性并验证模型。与传统点估计不同，贝叶斯方法将所有待估参数视为随机变量，通过观测数据更新先验认知，最终得到完整的后验分布，全面刻画参数在观测条件下的可信度。

在似然函数设定上，假设实验反射率与模型预测值的残差服从 Student-t 分布（自由度 $\nu=4$ ），以提高对异常点的鲁棒性。先验分布采用弱信息策略：主要物理参数（包括厚度）在物理合理区间内设均匀分布，Lorentz 振子的强度与宽度施加半正态收缩先验，噪声尺度参数采用半柯西分布。

数值实现上，使用仿射不变系综采样器进行 MCMC 采样。每个数据集采用 64 个行走者迭代 4000 步，舍弃前 1500 步作为 burn-in，以确保后验样本的平稳性和代表性，为不确定性评估和模型比较提供可靠依据。

5.2 检验结果与验证分析

在获得后验分布后，本文通过参数可信区间估计、后验预测检验（PPC）以及信息准则（WAIC 与 LOO）对所建立的模型进行系统验证。具体而言：

Step 1: 数据处理与初步分析

遵循 4.3 节中所构建的物理与数学模型，对实验光谱数据进行清洗与预处理。同时可初步观察光谱特性与可能的多光束干涉现象，为模型选择和参数初值设定提供依据。

Step 2: 点估计与拟合参数

高质量预处理数据基础上，采用确定性最小二乘法进行参数拟合。点估计提供了厚度的初步数值，还作为贝叶斯推断的初值，有助于加速 MCMC 收敛并提高采样效率。

Step 3: 后验分析与不确定度量

通过后验分布，可提取厚度的中位数及 95%可信区间（CI），实现对测量结果的不确定性量化。

表9 样品厚度点估计与后验可信区间

样品	入射角	4.3 节所得厚度 $d(\mu\text{m})$	95%CI(μm)
Si	10°	3.3885	3.2943 – 3.4606
Si	15°	3.3938	3.3039 – 3.4029
SiC	10°	7.0772	7.0774 – 7.2368
SiC	15°	7.0974	7.0975 – 7.1365

由表 9 可见，Si 样品的厚度点估计 d 均落在 MCMC 后验 95%可信区间内，表明模型对 Si 外延层的厚度测量高度可靠。对于 SiC 样品，厚度点估计略微偏离后验 95%可信区间的边界，但偏离程度极小。这可能是由于 SiC 样品多光束干涉效应相对较弱，使厚度拟合对实验数据的敏感性略有降低。

总体来看，该验证结果充分支持了本文所建立模型在厚度反演与不确定性量化方面的准确性与稳健性。

六、模型评估

6.1 模型的优点

1) 高精度拟合:

模型结合 Drude-Lorentz 模型、偏振与菲涅尔方程等物理规律，充分考虑光与晶格的共振效应，建立了多光束干涉物理模型。在 FFT+TRF 拟合算法的辅助下，实现了对实验光谱的高精度重现。

2) 数学与物理基础严谨:

所建立的干涉模型具有严格的数学推导与物理依据，有效消除了绘图和求解过程中的不确定性与潜在误差，保证了模型结果的可靠性和可解释性。

3) 贝叶斯推断支持:

通过贝叶斯方法进行参数后验分布估计，提供了模型参数的不确定性量化和可信区间，增强了模型的科学性和可推广性。

6.2 模型的不足

1) 计算复杂性:

多光束干涉模型结合贝叶斯推断时计算量较大，对计算资源和时间要求较高。

2) 潜在误差来源:

由于缺乏载流子浓度等实验数据，模型中折射率的估算存在一定误差，从而可能影响厚度计算的精确性。

6.3 模型的推广

1) 广泛的行业适用性:

本文建立的双光束干涉模型与多光束干涉模型，以及贝叶斯推断模型，都不依赖于具体的材料，只要有对应的实验数据，就可以完成拟合并计算外延层厚度。

2) 算法可扩展性:

贝叶斯推断框架和 FFT+TRF 拟合算法可与其他光学模型结合，例如椭偏仪测量或反射吸收光谱分析，实现更多物理参数的联合反演，为复杂多层薄膜或异质结构的表征提供可行方案。

3) 科研与工业应用:

该模型不仅可用于学术研究中的外延层厚度测量，也可应用于半导体晶圆制造过程的在线监测、质量控制及工艺优化，具有显著的工程推广价值。

参考文献

- [1] Cherkassky, A. (1998). Metrology of very thin silicon epitaxial films.
- [2] Li, Z.-Y., et al. (2010). Chinese Phys. Lett., 27, 068103.
- [3] Tong, Z., Liu, L., Li, L., & Bao, H. (2018). Temperature-dependent infrared optical properties of 3C-, 4H- and 6H-SiC. Physica B: Condensed Matter, 537, 194–201.
- [4] 赵凯华, & 钟锡华. (2018). 光学(重排本). 北京: 高等教育出版.
- [5] Kittel, C. (2004). Introduction to Solid State Physics (8th ed.). Wiley.
- [6] Shaffer, P. T. B. (1971). Refractive index, dispersion, and birefringence of silicon carbide poly-types.
- [7]] DeepSeek, DeepSeek-V3.1-0831, 深度求索(DeepSeek), 2025-09-06

附录 A 支撑材料文件列表

表10 文件列表

文件名	说明
q2.py	SiC 外延层厚度直接计算
q2_nihe.py	双光束干涉物理模型拟合 SiC
q3_fft.py	使用 FFT 绘制频谱图
q3_nihe.py	多光束干涉物理模型与厚度计算
q3_bys.py	贝叶斯拓展
AI 工具使用情况.pdf	与 AI 关键交互记录

附录 B 参数后验汇总

表11 Si 参数后验汇总

θ_1	参数	中文说明	2.5%	中位数	97.5%
10°	d	外延层厚度(μm)	3.294301	3.301893	3.460551
	ϵ_∞	高频介电常数	11.960165	13.171050	13.253127
	ν_p	Drude 模型等离子体频率 (cm^{-1})	106.993985	335.370080	416.173345
	γ_d	Drude 阻尼系数 (cm^{-1})	1731.015634	1935.750395	1998.272220
	S_1	第一个 Lorentz 振子强度	6.875476	7.582994	7.682677
	ν_{01}	第一个 Lorentz 振子共振频率(cm^{-1})	727.525845	729.184445	730.876963
	γ_1	第一个 Lorentz 阻尼(cm^{-1})	66.570598	69.187814	72.439494
	S_2	第二个 Lorentz 振子强度	1.395156	1.548211	1.597358
	ν_{02}	第二个 Lorentz 共振频率(cm^{-1})	2654.295122	2704.697720	2740.223529
	γ_2	第二个 Lorentz 阻尼(cm^{-1})	2717.409696	2934.297722	2996.890823

	p_{frac}	p 分量占比	0.001370	0.033931	0.198439
	σ_{nm}	表面粗糙度(nm)	0.039382	1.141199	26.001713
	σ_d	厚度分布标准差 (μm)	0.120996	0.125027	0.130803
	σ_{ang}	入射角分布标准差($^\circ$)	0.584433	1.475862	1.710517
	$\text{FWHM}_{\text{inst}}$	仪器分辨率(cm^{-1})	0.062448	0.422048	1.794679
	scale	光谱缩放系数	0.995800	1.021537	1.045203
	offset	光谱偏置	-5.892828	-5.023217	-3.599259
	slope	光谱线性斜率 ($\%/ \text{cm}^{-1}$)	-0.000085	0.000128	0.000548
	$\log \sigma$	后验分布 log 标准差	0.679560	0.720998	0.807890
	WAIC	预测误差指标	43443.3 - 458	43443.3	43443.3 + 458
15°	d	外延层厚度(μm)	3.303884	3.343734	3.402925
	ε_∞	高频介电常数	1.960926	7.434644	7.927171
	ν_p	Drude 模型等离子体频率(cm^{-1})	8.920449	44.412260	127.396268
	γ_d	Drude 阻尼系数 (cm^{-1})	14.175169	1203.798992	1956.675139
	S_1	第一个 Lorentz 振子强度	1.998307	2.226711	2.463733
	ν_{01}	第一个 Lorentz 振子共振频率(cm^{-1})	454.511303	463.837500	473.494493
	γ_1	第一个 Lorentz 阻尼(cm^{-1})	6.586821	10.420903	17.691344
	S_2	第二个 Lorentz 振子强度	3.527923	4.038251	9.595990
	ν_{02}	第二个 Lorentz 共振频率(cm^{-1})	4380.163548	4865.155458	7768.986615

γ_2	第二个 Lorentz 阻尼(cm^{-1})	2117.116939	2691.580043	2974.468695
p_{frac}	p 分量占比	0.015541	0.157050	0.415899
σ_{nm}	表面粗糙度(nm)	24.915482	45.871609	49.776117
σ_d	厚度分布标准差(μm)	0.137174	0.147588	0.158195
σ_{ang}	入射角分布标准差($^\circ$)	0.378951	1.209525	1.943321
$\text{FWHM}_{\text{inst}}$	仪器分辨率(cm^{-1})	0.051287	0.557464	6.659700
scale	光谱缩放系数	1.113932	1.147248	1.194612
offset	光谱偏置	-5.407415	-3.624609	-2.382692
slope	光谱线性斜率($\%/ \text{cm}^{-1}$)	-0.000996	-0.000933	-0.000801
$\log \sigma$	后验分布 log 标准差	0.774225	0.932010	1.281140
WAIC	预测误差指标	45038.2 - 419.9	45038.2	45038.2 + 419.9

表12 SiC 参数后验汇总

θ_1	参数	中文说明	2%	中位数	97%
10°	d	外延层厚度 (μm)	7.077432	7.089331	7.236849
	ε_∞	高频介电常数	7.091334	7.314704	7.351693
	ν_p	Drude 模型等离子体频率 (cm^{-1})	1219.509258	1308.100485	1383.039143
	γ_d	Drude 阻尼系数 (cm^{-1})	1.250776	6.932667	25.680513
	S_1	第一个 Lorentz 振子强度	9.529417	9.905374	9.996472
	ν_{01}	第一个 Lorentz 振子共振频率 (cm^{-1})	695.564017	715.638475	729.380541

	γ_1	第一个 Lorentz 阻尼(cm^{-1})	1991.241521	1997.773901	1999.894786
	S_2	第二个 Lorentz 振子强度	0.178609	0.189426	0.197960
	ν_{02}	第二个 Lorentz 共振频率(cm^{-1})	3901.337057	4107.842200	4309.074603
	γ_2	第二个 Lorentz 阻尼(cm^{-1})	2988.291031	2996.842930	2999.883809
	p_{frac}	p 分量占比	0.908529	0.978186	0.998694
	σ_{nm}	表面粗糙度(nm)	0.424231	3.227190	6.446410
	σ_d	厚度分布标准差 (μm)	0.199456	0.199906	0.199997
	σ_{ang}	入射角分布标准差($^\circ$)	1.341982	1.710958	1.977018
	$\text{FWHM}_{\text{inst}}$	仪器分辨率(cm^{-1})	0.027326	0.270883	0.391594
	scale	光谱缩放系数	0.850037	0.850983	0.854887
	offset	光谱偏置	0.716906	0.816610	1.437937
	slope	光谱线性斜率 ($\%/ \text{cm}^{-1}$)	0.000123	0.000131	0.000138
	$\log \sigma$	后验分布 log 标准差	-3.141006	-3.101970	-3.045765
	WAIC	预测误差指标	-12122.7 - 99.5	-12122.7	-12122.7 + 99.5
15°	d (μm)	外延层厚度 (μm)	7.097476	7.101650	7.136540
	ϵ_∞	高频介电常数	6.682638	6.723824	6.775549
	vp (Drude)	Drude 模型等离子体频率 (cm^{-1})	895.743275	927.164628	961.984344
	γ (Drude)	Drude 阻尼系数 (cm^{-1})	6.776901	99.426390	264.562828
	L1 S	第一个 Lorentz 振子强度	9.623281	9.900396	9.995321

L1 ν_0	第一个 Lorentz 振子共振频率 (cm^{-1})	691.291782	702.820236	714.958903
L1 γ	第一个 Lorentz 阻尼(cm^{-1})	1978.010587	1993.184663	1999.683664
L2 S	第二个 Lorentz 振子强度	0.386391	0.430561	0.454095
L2 ν_0	第二个 Lorentz 共振频率(cm^{-1})	5639.913264	5842.524876	5950.495023
L2 γ	第二个 Lorentz 阻尼(cm^{-1})	2918.143833	2985.742178	2999.676510
pfrac	p 分量占比	0.933582	0.984584	0.999376
σ (nm)	表面粗糙度(nm)	0.225976	6.622865	13.121381
σ_d (μm)	厚度分布标准差 (μm)	0.199207	0.199837	0.199996
σ_{ang} ($^\circ$)	入射角分布标准 差($^\circ$)	1.774849	1.958531	1.998180
$\text{FWHM}_{\text{inst}}$	仪器分辨率(cm^{-1})	0.007927	0.179541	0.405665
scale	光谱缩放系数	0.850150	0.856563	0.880845
offset	光谱偏置	1.571736	1.956442	2.110678
slope	光谱线性斜率 ($\%/ \text{cm}^{-1}$)	0.000229	0.000243	0.000257
$\log \sigma$	后验分布 log 标 准差	-2.880547	-2.810695	-2.697880
WAIC	预测误差指标	-9188.9 - 159.6	-9188.9	-9188.9 + 159.6

附录 C 支撑材料所有代码

附录 C-1

SiC 外延层厚度直接计算与波峰绘制代码

```
'''
根据问题 1 推导出的数学公式和附件 1、附件 2 中的数据，直接计算外延层厚度 d，并绘制波峰图。
'''
import pandas as pd
import matplotlib.pyplot as plt
import os
from scipy.signal import find_peaks
import numpy as np
import re
from math import sin, radians, sqrt
base_directory = r'D:\数模国赛\B 题\附件'
files_to_plot = {
    '附件 1.xlsx': 'SiC 10° incidence angle',
    '附件 2.xlsx': 'SiC 15° incidence angle',
}
def calculate_peak_distance(wavenumbers, reflectances, peak_indices):
    """计算指定波峰之间的横轴差值"""
    if len(peak_indices) < 2:
        return None, "检测到的波峰数量不足 2 个"

    peak_wavenumbers = [wavenumbers[i] for i in peak_indices]

    peak_diffs = []
    for i in range(len(peak_wavenumbers) - 1):
        diff = abs(peak_wavenumbers[i+1] - peak_wavenumbers[i])
        peak_diffs.append((peak_wavenumbers[i], peak_wavenumbers[i+1], diff))

    return peak_diffs, "计算成功"
def calculate_d_value(k1, k2, theta_degrees):
    """
    根据公式计算 d 值
    k1, k2: 相邻两个波峰的波数值
    theta_degrees: 入射角 (度)
    注意: k1 和 k2 需要乘以 10^-4
    """
    k1_scaled = k1 * 1e-4
    k2_scaled = k2 * 1e-4

    # 计算入射角的正弦值
    sin_theta = sin(radians(theta_degrees))

    # 计算公式中的两个平方根项
    term1 = sqrt(4.588*k1_scaled**2 + 1.975*k1_scaled**2/(1-0.116*k1_scaled**2) - sin_theta**2*k1_scaled**2)
    term2 = sqrt(4.588*k2_scaled**2 + 1.975*k2_scaled**2/(1-0.116*k2_scaled**2) - sin_theta**2*k2_scaled**2)
```

```

# 计算d 值
d = 0.5 * 1 / (term2 - term1)
return d
print("开始生成图表并分析波峰...")
for file_name, description in files_to_plot.items():

    file_path = os.path.join(base_directory, file_name)

    if not os.path.exists(file_path):
        print(f"警告: 文件 '{file_path}' 未找到, 已跳过。")
        continue
    try:
        theta_match = re.search(r'(\d+)°', description)
        if theta_match:
            theta_degrees = float(theta_match.group(1))
            print(f"从文件描述中提取到入射角: {theta_degrees}°")
        else:
            print(f"无法从描述中提取入射角: {description}")
            continue

        # 读取Excel 文件
        df = pd.read_excel(file_path, skiprows=1, engine='openpyxl')

        # 处理数据格式
        if len(df.columns) != 2:
            df = pd.read_excel(file_path, header=None, engine='openpyxl')

        if len(df.columns) >= 2:
            df = df.iloc[:, :2]
            df.columns = ['Wavenumber', 'Reflectance']

            df['Wavenumber'] = pd.to_numeric(df['Wavenumber'], errors='co-
erces')
            df['Reflectance'] = pd.to_numeric(df['Reflectance'], er-
rors='coerce')
            df = df.dropna()

            wavenumbers = df['Wavenumber'].values
            reflectances = df['Reflectance'].values

            # 检测波峰
            peaks, _ = find_peaks(reflectances,
                                   height=10,
                                   distance=325,
                                   prominence=0.1)

            # 绘制光谱图并标记波峰
            plt.figure(figsize=(12, 6))
            plt.plot(wavenumbers, reflectances, color='royalblue', lin-
ewidth=1.5, label='spectral curve')
            plt.scatter(wavenumbers[peaks], reflec-
tances[peaks], color='red', s=50, label='detected peaks')

            # 突出显示波数大于2000 的波峰

```

```

        high_wavenumber_peaks = [i for i in peaks if wave-
numbers[i] > 2000]
        plt.scatter(wavenumbers[high_wavenumber_peaks], reflec-
tances[high_wavenumber_peaks],
                    color='green', s=70, facecolors='none', la-
bel='peaks with wavenumber > 2000')

        plt.title(f'Interferometric spectrogram - {descrip-
tion}', fontsize=16)
        plt.xlabel('wavenumber (1/cm)', fontsize=12)
        plt.ylabel('reflectance (%)', fontsize=12)
        plt.legend()
        plt.grid(True)
        plt.show()

        # 计算所有波峰之间的差值
        peak_diffs, message = calculate_peak_distance(wavenumbers, re-
flectances, peaks)

        print(f"\n{description} 的波峰分析结果:")
        print(f"共检测到 {len(peaks)} 个波峰")

        if peak_diffs:
            print("所有波峰位置及相邻差值 (cm-1):")
            for i, (peak1, peak2, diff) in enumerate(peak_diffs):
                print(f" 波峰{i+1} ({peak1:.2f}) 与 波峰
{i+2} ({peak2:.2f}) 的差值: {diff:.2f}")

            # 筛选波数大于 2000 的波峰
            high_wavenumber_peak_values = [wave-
numbers[i] for i in peaks if wavenumbers[i] > 2000]
            print(f"\n波数大于 2000 的波峰数量: {len(high_wave-
number_peak_values)}")

            d_total = 0.0
            if len(high_wavenumber_peak_values) >= 2:
                print("波数大于 2000 的相邻波峰及 d 值计算结果:")
                for i in range(len(high_wavenumber_peak_values) - 1):
                    k1 = high_wavenumber_peak_values[i]
                    k2 = high_wavenumber_peak_values[i+1]
                    d = calculate_d_value(k1, k2, theta_degrees)
                    d_total += d
                    print(f" 波峰 {k1:.2f} 与 {k2:.2f} 之间的 d
值: {d:.6f}")
            else:
                print("波数大于 2000 的波峰数量不足 2 个, 无法计算 d 值")
                print(f"d 平均值: {d_total/(len(high_wavenumber_peak_values)-
1):.6f}\n")
            else:
                print(f"错误: 文件 '{file_path}' 格式不正确, 没有足够的列。")
        except Exception as e:
            print(f"处理文件 '{file_path}' 时发生错误: {str(e)}")
    print("\n所有图表及波峰分析已完成。")

```

附录 C-2

双光束干涉物理模型拟合 SiC

```
'''
使用建立的双光束干涉物理模型，先试用 FFT 方法估计外延层厚度 d 的初值，然后使用 TRF 算
法进行精修拟合
'''
import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
from scipy.fft import fft, fftfreq
from scipy.optimize import least_squares
import matplotlib.pyplot as plt
import os
# --- 1. 全局设置 ---
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
# --- 2. 数据加载函数 ---
def load_data(filepath):
    try:
        df = pd.read_excel(filepath, header=None, names=['wavenumber', 're-
flectance'])
        df['wavenumber'] = pd.to_numeric(df['wavenumber'], errors='coerce')
        df['reflectance'] = pd.to_numeric(df['reflectance'], errors='co-
erce')
        df_clean = df.dropna(axis=0, how='any')
        df_clean = df_clean[(df_clean['wavenumber'] > 0) & (df_clean['re-
flectance'] >= 0)]
        if len(df_clean) == 0: return None, None
        print(f"成功加载并清洗数据: 共 {len(df_clean)} 个有效数据点")
        return df_clean['wavenumber'].values, df_clean['reflectance'].values
    except FileNotFoundError:
        print(f"错误: 文件 '{filepath}' 未找到。当前工作目录: {os.getcwd()}")
        return None, None
    except Exception as e:
        print(f"数据加载异常: {str(e)}")
        return None, None
# --- 3. FFT 初始估计算法 ---
def estimate_thickness_fft(wavenumber, reflectance):
    wavenumber_uniform = np.linspace(wavenumber.min(), wavenumber.max(), num=len(wavenumber) * 4)
    interp_func = interp1d(wavenumber, reflectance, kind='cubic')
    reflectance_uniform = interp_func(wavenumber_uniform)
    reflectance_fft_input = reflectance_uniform - np.mean(reflectance_uniform)
    N = len(reflectance_fft_input)
    yf = fft(reflectance_fft_input)
    xf_cm = fftfreq(N, d=wavenumber_uniform[1] - wavenumber_uniform[0])
    xf_um = xf_cm * 10000

    min_opd_search_um = 20.0
```

```

max_opd_search_um = 60.0
mask = (xf_um > min_opd_search_um) & (xf_um < max_opd_search_um)

peak_idx = np.argmax(np.abs(yf[mask]))
opd_peak_um = xf_um[mask][peak_idx]

return opd_peak_um, (xf_um[xf_um > 0], np.abs(yf[xf_um > 0]))
# --- 4. 使用 TRF 的拟合算法 ---
def fit_excellent_model(wavenumber, reflectance, angle_deg, opd_initial):

    def residuals(params, wavenumber, reflectance, angle_deg):
        d_um, n_A, n_B, base_A0, base_A1, amp_B = params
        n_of_wavenumber = n_A + n_B / (wavenumber**2)
        angle_rad = np.deg2rad(angle_deg)
        opd = 2 * (d_um / 10000) * np.sqrt(n_of_wavenumber**2 - np.sin(an-
gle_rad)**2)
        delta = 2 * np.pi * opd * wavenumber
        reflectance_theory = (base_A0 + base_A1 * wave-
number) - amp_B * np.cos(delta)
        return reflectance - reflectance_theory

    # 1. 计算数据的基本统计特性
    data_mean = np.mean(reflectance)
    data_min = np.min(reflectance)
    data_max = np.max(reflectance)
    data_amplitude = (data_max - data_min) / 2

    # 2. 设定初始猜测值 (与之前相同)
    n1_initial_guess = 2.55
    # 注意: 这里的 d_initial 计算也需要 n1 的猜测值, 所以要先算出来
    d_initial = opd_initial / (2 * np.sqrt(n1_ini-
tial_guess**2 - np.sin(np.deg2rad(angle_deg))**2))

    initial_params = [
        d_initial,
        n1_initial_guess,
        0,
        data_mean, # 基线截距初值
        0,
        data_amplitude # 振幅初值
    ]

    # 3. 设定更稳健、更具适应性的边界
    bounds = (
        # 下界
        [
            d_initial * 0.95, # d 的下界
            2.4, # n_A 的下界
            -1e7, # n_B 的下界
            data_min, # 【修正】基线截距的下界, 不能低于数据最小值
            -0.01, # base_A1 的下界
            0 # 振幅的下界
        ],
        # 上界
        [

```



```

        d_initial * 1.05,      # d 的上界
        2.7,                  # n_A 的上界
        1e7,                  # n_B 的上界
        data_max,             # 【修正】基线截距的上界，不能高于数据最大值
        0.01,                 # base_A1 的上界
        data_amplitude * 2.0  # 【修正】振幅的上界，设为数据振幅的 2 倍，更
合理
    ]
)
# 4. 执行 TRF 优化
result = least_squares(
    residuals,
    initial_params,
    args=(wavenumber, reflectance, angle_deg),
    bounds=bounds,
    method='trf',
    verbose=0
)

return result.x, result.cost * 2
# --- 5. 主程序 ---
def main():
    files_to_process = {
        '附件 1 (10°)': {'path': './B 题/附件/附件 1.xlsx', 'angle': 10},
        '附件 2 (15°)': {'path': './B 题/附件/附件 2.xlsx', 'angle': 15}
    }

    WAVENUMBER_MIN = 2000
    WAVENUMBER_MAX = 4000
    results = {}
    for name, info in files_to_process.items():
        print(f"--- 正在处理: {name} ---")
        wavenumber_raw, reflectance_raw = load_data(info['path'])
        if wavenumber_raw is None: continue
        mask = (wavenumber_raw >= WAVENUMBER_MIN) & (wavenumber_raw <= WAVENUMBER_MAX)
        wavenumber, reflectance = wavenumber_raw[mask], reflectance_raw[mask]
        print(f"数据筛选完成: 只分析波数在 [{WAVENUMBER_MIN}, {WAVENUMBER_MAX}] cm-1 的数据。")
        # 步骤 1: FFT 估计光学厚度 OPD 的初值
        opd_initial, fft_data = estimate_thickness_fft(wavenumber, reflectance)
        print(f"步骤 1: FFT 初始估计光学厚度 OPD = {opd_initial:.4f} μm")
        # 步骤 2: 使用双光束干涉模型进行精修拟合
        print("步骤 2: 正在执行 TRF 精修拟合 (含色散模型)...")
        best_params, final_error = fit_excellent_model(wavenumber, reflectance, info['angle'], opd_initial)
        d_final, nA_f, nB_f, *_ = best_params

        print("步骤 2: 优化完成。")
        print(f"最终计算厚度 d = {d_final:.4f} μm")
        print(f"计算出的折射率模型: n(v) = {nA_f:.4f} + {nB_f:.1e} / v2")

```

```

    print(f" 最终拟合误差 (SSR) = {final_error:.2f}")
    print(f" 最终均方误差 (MSE) = {final_error / len(mask):.4f}")

    re-
sults[name] = {'params': best_params, 'fft_data': fft_data, 'raw_data': (wav
enumber_raw, reflectance_raw)}
    print("-" * 60 + "\n")
    # --- 结果可视化与分析 ---
    print("\n--- 正在生成可视化图表 ---")
    fig, axes = plt.subplots(len(files_to_process), 2, fig-
size=(18, 12), gridspec_kw={'width_ratios': [1, 2]})
    if len(files_to_process) == 1:
        axes = np.array([axes])

    fig.suptitle('双光束干涉模型拟合结果与分析', fontsize=20)
    for i, (name, res) in enumerate(results.items()):
        angle = files_to_process[name]['angle']
        wavenumber_raw, reflectance_raw = res['raw_data']
        params = res['params']
        d_final, nA_f, nB_f, base_A0_f, base_A1_f, amp_B_f = params

        # ----- 左侧图: FFT 频谱分析 -----
        ax_fft = axes[i, 0]
        fft_x, fft_y = res['fft_data']

        # 使用平均波数计算一个代表性的折射率, 用于估算 OPD
        wavenumber_fit_region = wavenumber_raw[(wavenumber_raw >= WAVE-
NUMBER_MIN) & (wavenumber_raw <= WAVENUMBER_MAX)]
        n_final_example = nA_f + nB_f / (np.mean(wavenumber_fit_region)**2)
        opd_final = 2 * d_final * np.sqrt(n_final_exam-
ple**2 - np.sin(np.deg2rad(angle))**2)
        ax_fft.plot(fft_x, fft_y, label='FFT 幅值')
        ax_fft.axvline(opd_final, color='r', linestyle='--', label=f'最终
OPD ≈ {opd_final:.2f}μm')
        ax_fft.set_title(f'{name} - FFT 频谱分析', fontsize=14)
        ax_fft.set_xlabel('光学厚度 (μm)', fontsize=12)
        ax_fft.set_ylabel('FFT 幅值', fontsize=12)
        ax_fft.set_xlim(20, 60)
        ax_fft.legend()
        ax_fft.grid(True, linestyle='--', alpha=0.6)
        # ----- 右侧图: 拟合结果与残差分析 -----
        sub_gridspec = axes[i, 1].get_subplotspec().subgrid-
spec(3, 1, hspace=0.1)
        axes[i, 1].set_axis_off()
        ax_fit = fig.add_subplot(sub_gridspec[0:2, 0])
        ax_res = fig.add_subplot(sub_gridspec[2, 0], sharex=ax_fit)
        # 计算全谱的理论曲线
        n_of_wavenumber_raw = nA_f + nB_f / (wavenumber_raw**2)
        opd_raw = 2 * (d_final / 10000) * np.sqrt(n_of_wave-
number_raw**2 - np.sin(np.deg2rad(angle))**2)
        delta_raw = 2 * np.pi * opd_raw * wavenumber_raw
        reflectance_fit_raw = (base_A0_f + base_A1_f * wave-
number_raw) - amp_B_f * np.cos(delta_raw)

```

```

# 1. 绘制拟合图
ax_fit.plot(wavenumber_raw, reflectance_raw, label='实验数据', lw=1.5, alpha=0.8)
ax_fit.plot(wavenumber_raw, reflectance_fit_raw, label='模型拟合曲线', color='red', linestyle='--', lw=2)
ax_fit.axvspan(WAVENUMBER_MIN, WAVENUMBER_MAX, color='green', alpha=0.1, label='拟合区域')
ax_fit.set_title(f'{name} - 全谱拟合结果 (d = {d_fit:.4f} μm)', fontsize=14)
ax_fit.set_ylabel('反射率 (%)', fontsize=12)
ax_fit.legend()
ax_fit.grid(True, linestyle='--', alpha=0.6)
plt.setp(ax_fit.get_xticklabels(), visible=False)

# 2. 计算并绘制残差图
residuals_raw = reflectance_raw - reflectance_fit_raw
ax_res.plot(wavenumber_raw, residuals_raw, color='green', lw=1)
ax_res.axhline(0, color='black', linestyle='--', lw=1)
ax_res.set_xlabel('波数', fontsize=12)
ax_res.set_ylabel('残差', fontsize=12)
ax_res.set_ylim(-1.5, 1.5)
ax_res.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# --- 可靠性分析 ---
d1 = results.get('附件 1 (10°)', {}).get('params', [np.nan])[0]
d2 = results.get('附件 2 (15°)', {}).get('params', [np.nan])[0]
print("\n--- 结果可靠性分析 ---")
print(f"入射角 10° 时的计算厚度: {d1:.4f} μm")
print(f"入射角 15° 时的计算厚度: {d2:.4f} μm")
difference = np.abs(d1 - d2); relative_diff = (difference / np.mean([d1, d2])) * 100
print(f"两次测量结果的绝对差值: {difference:.4f} μm")
print(f"两次测量结果的相对差值: {relative_diff:.4f} %")
print(f"\n结论: 结果高度一致。最终报告厚度为平均值: {d_avg:.4f} μm。")
if __name__ == '__main__':
    main()

```

附录 C-3

使用 FFT 绘制频谱图

```

'''
使用 FFT 方法分析四个附件的反射率数据，绘制频谱图并标注峰值位置，确定是否有多光束干涉现象
'''
import os
import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
from numpy.fft import rfft, rfftfreq
import matplotlib.pyplot as plt

```

```

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['figure.dpi'] = 120
# ----- 读取与清洗数据 -----
def load_data(filepath):
    """
    从 Excel 读取两列数据：波数(cm-1) 和 反射率(%) 并清洗排序
    """
    try:
        df = pd.read_excel(filepath, header=None, names=['wavenumber', 'reflectance'])
    except FileNotFoundError:
        print(f"错误：文件 '{filepath}' 未找到。")
        return None, None
    # 数值化 + 清洗
    df['wavenumber'] = pd.to_numeric(df['wavenumber'], errors='coerce')
    df['reflectance'] = pd.to_numeric(df['reflectance'], errors='coerce')
    df = df.dropna()
    df = df[(df['wavenumber'] > 0) & (df['reflectance'] >= 0)]
    if df.empty:
        print(f"错误：文件 '{filepath}' 数据为空或无效。")
        return None, None
    df = df.sort_values('wavenumber').reset_index(drop=True)
    return df['wavenumber'].values, df['reflectance'].values
# ----- 计算 FFT 频谱（正频率） -----
def fft_spectrum(wavenumber, reflectance, upsample=4):
    """
    对反射率(随波数)进行插值、去直流后做 rFFT。
    返回：
        xf_um: 以 μm 为单位的“光学厚度”刻度
        amp: FFT 幅值
    """
    # 均匀插值（波数域）
    x_uniform = np.linspace(wavenumber.min(), wavenumber.max(), num=len(wavenumber) * upsample)
    y_uniform = interp1d(wavenumber, reflectance, kind='cubic', fill_value='extrapolate')(x_uniform)
    # 去直流
    y_centered = y_uniform - np.mean(y_uniform)
    # rFFT（仅正频）
    N = len(y_centered)
    dx = x_uniform[1] - x_uniform[0]
    yf = rfft(y_centered)
    xf_cm = rfftfreq(N, d=dx) # 单位: cycles per cm-1
    xf_um = xf_cm * 10000.0 # 转为 μm（等效 OPD 轴）
    amp = np.abs(yf)
    return xf_um, amp
# ----- 寻找 OPD 峰值 -----
def find_peak_opd(xf_um, amp, opd_range=None):
    """
    在指定的 OPD 范围内找 FFT 幅值峰值。
    若未提供范围或范围内无数据，则在所有正频中寻找最大峰。
    """
    if opd_range is not None:

```

```

        mask = (xf_um >= opd_range[0]) & (xf_um <= opd_range[1])
        if np.any(mask):
            idx = np.argmax(amp[mask])
            return xf_um[mask][idx]
    pos_mask = xf_um > 0
    if not np.any(pos_mask):
        return None
    idx = np.argmax(amp[pos_mask])
    return xf_um[pos_mask][idx]
# ----- 绘图 -----
def plot_fft_single(name, xf_um, amp, opd_peak=None, save-
path=None, xlim=(0, 80)):
    """
    绘制单张 FFT 频谱图
    """
    fig, ax = plt.subplots(figsize=(7.5, 5.2))
    ax.plot(xf_um, amp, color='tab:blue', lw=1.8)
    if opd_peak is not None:
        ax.axvline(opd_peak, color='crimson', ls='--', lw=1.5)
        ax.text(
            0.98, 0.92, f"峰值 OPD ≈ {opd_peak:.2f} μm",
            transform=ax.transAxes, ha='right', va='top', fontsize=11,
            bbox=dict(facecolor='white', edgecolor='none', al-
pha=0.8, pad=2.5)
        )
    ax.set_title(f"{name} - FFT 频谱", fontsize=13)
    ax.set_xlabel("光学厚度 (μm)", fontsize=12)
    ax.set_ylabel("FFT 幅值 (a.u.)", fontsize=12)
    ax.grid(True, linestyle=':', alpha=0.6)
    if xlim is not None:
        ax.set_xlim(*xlim)
    ax.set_ylim(bottom=0)
    fig.tight_layout()
    if savepath:
        fig.savefig(savepath, dpi=200, bbox_inches='tight')
    return fig, ax
# ----- 主流程: 输出四张 FFT 图 -----
def main():
    # 数据与范围设置
    files = {
        '附件 3 (Si, 10°)': {'path': './B 题/附件/附件
3.xlsx', 'w_range': (500, 4000), 'opd_range': (20, 30)},
        '附件 4 (Si, 15°)': {'path': './B 题/附件/附件
4.xlsx', 'w_range': (500, 4000), 'opd_range': (20, 30)},
        '附件 1 (SiC, 10°)': {'path': './B 题/附件/附件
1.xlsx', 'w_range': (1500, 4000), 'opd_range': (35, 45)},
        '附件 2 (SiC, 15°)': {'path': './B 题/附件/附件
2.xlsx', 'w_range': (1500, 4000), 'opd_range': (35, 45)},
    }
    out_dir = "./输出_FFT 图"
    os.makedirs(out_dir, exist_ok=True)
    for name, info in files.items():
        wavenumber, reflectance = load_data(info['path'])
        if wavenumber is None:

```

```

        continue
    # 按需截取分析波段
    mask = (wavenumber >= info['w_range'][0]) & (wave-
number <= info['w_range'][1])
    wn, rf = wavenumber[mask], reflectance[mask]
    if len(wn) < 10:
        print(f"警告: {name} 可用数据点太少, 跳过。")
        continue
    # 计算 FFT 频谱
    xf_um, amp = fft_spectrum(wn, rf, upsample=4)
    # 寻找 OPD 峰
    opd_peak = find_peak_opd(xf_um, amp, opd_range=info['opd_range'])
    # 绘制与保存 (单张)
    safe_name = name.replace('/', '_').replace(' ', '_')
    savepath = os.path.join(out_dir, f"FFT_{safe_name}.png")
    plot_fft_single(name, xf_um, amp, opd_peak=opd_peak, savepath=save-
path, xlim=(0, 80))
    print(f"已保存: {savepath}")
    plt.show()
if __name__ == "__main__":
    main()

```

附录 C-4

多光束干涉物理模型与厚度计算

```

import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
from scipy.fft import fft, fftfreq
from scipy.optimize import least_squares
import matplotlib.pyplot as plt
from numpy.polynomial.hermite import hermgauss
import sys, time # 用于实时输出
# SciPy least_squares 的内部“函数评估上限”
MAX_NFEV_LS = 1000 # 可改成 3000/8000 等
def estimate_calls_budget(max_nfev, n_params, jac_kind='2-point'):
    """
    估算“残差调用预算”，用于进度条分母
    2-point: ~ (1 + p) * max_nfev
    3-point: ~ (1 + 2p) * max_nfev
    cs:      ~ (1 + p) * max_nfev (complex-step)
    """
    if jac_kind == '3-point':
        k = 1 + 2*n_params
    else: # '2-point' 或 'cs'
        k = 1 + n_params
    return int(max_nfev * k)
# -----
# 0. 实时进度输出配置 (可调)
# -----
REALTIME_PROGRESS = True # 关闭实时输出请设为 False
PROGRESS_EVERY = 60 # 每多少次函数评估打印一次进度
TIME_THROTTLE = 0.25 # 限流: 两次打印之间至少间隔秒数

```

```

# -----
# 1. Matplotlib 全局设置
# -----
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
# -----
# 辅助: 简易进度打印器
# -----
class IterPrinter:
    def __init__(self, label, max_nfev, every=100, time_throttle=0.25, enable=True):
        self.label = label
        self.max_nfev = max(1, int(max_nfev))
        self.every = max(1, int(every))
        self.enable = enable
        self.reset()
        self.time_throttle = time_throttle
    def reset(self):
        self.nfev = 0
        self.best_ssr = np.inf
        self.last_print_nfev = -1
        self.t0 = time.perf_counter()
        self.last_t = 0.0
    def update(self, res_vec, params=None):
        if not self.enable:
            return
        self.nfev += 1
        ssr = float(np.dot(res_vec, res_vec))
        if ssr < self.best_ssr:
            self.best_ssr = ssr
        # 控制输出频率
        now = time.perf_counter()
        do_print = False
        if self.nfev == 1 or self.nfev >= self.max_nfev:
            do_print = True
        elif (self.nfev - self.last_print_nfev) >= self.every and (now - self.last_t) >= self.time_throttle:
            do_print = True
        if do_print:
            pct = min(100.0, 100.0 * self.nfev / self.max_nfev)
            d_str = ""
            if params is not None and len(params) > 0:
                try:
                    d_str = f" | d≈{float(params[0]):.4f} μm"
                except Exception:
                    d_str = ""
            msg = f"\r[{self.label}] nfev={self.nfev}/{self.max_nfev} ({pct:5.1f}%) | 当前 SSR={ssr:.3e}{d_str}"
            sys.stdout.write(msg)
            sys.stdout.flush()
            self.last_print_nfev = self.nfev
            self.last_t = now
    def close_line(self):
        if not self.enable:

```

```

        return
    sys.stdout.write("\n")
    sys.stdout.flush()
# -----
# 2. 数据加载
# -----
def load_data(filepath):
    try:
        df = pd.read_excel(filepath, header=None, names=['wavenumber', 'reflectance'])
        df['wavenumber'] = pd.to_numeric(df['wavenumber'], errors='coerce')
        df['reflectance'] = pd.to_numeric(df['reflectance'], errors='coerce')
        df_clean = df.dropna(axis=0, how='any')
        df_clean = df_clean[(df_clean['wavenumber'] > 0) & (df_clean['reflectance'] >= 0)]
        if len(df_clean) == 0:
            return None, None
        print(f"成功加载并清洗数据: 共 {len(df_clean)} 个有效数据点")
        return df_clean['wavenumber'].values, df_clean['reflectance'].values
    except FileNotFoundError:
        print(f"错误: 文件 '{filepath}' 未找到。")
        return None, None
# -----
# 3. FFT 初始 OPD 估计
# -----
def estimate_opd_fft(wavenumber, reflectance, opd_range=(20, 60)):
    wu = np.linspace(wavenumber.min(), wavenumber.max(), num=len(wavenumber) * 4)
    interp_func = interp1d(wavenumber, reflectance, kind='cubic')
    ru = interp_func(wu)
    x = ru - np.mean(ru)
    N = len(x)
    yf = fft(x)
    xf_cm = fftfreq(N, d=wu[1] - wu[0])
    xf_um = xf_cm * 10000
    mask = (xf_um > opd_range[0]) & (xf_um < opd_range[1])
    if not np.any(mask):
        return opd_range[0], (xf_um[xf_um > 0], np.abs(yf[xf_um > 0]))
    peak_idx = np.argmax(np.abs(yf[mask]))
    opd_peak_um = xf_um[mask][peak_idx]
    return opd_peak_um, (xf_um[xf_um > 0], np.abs(yf[xf_um > 0]))
# -----
# 4. 物理模型: Drude-Lorentz + s/p + 平均 + 仪器卷积
# -----
def drude_lorentz_eps(wavenumber, eps_inf, drude, lorentz_list):
    nu = np.asarray(wavenumber, dtype=float)
    eps = eps_inf + 0j
    if drude is not None:
        nu_p, gamma_d = drude
        if nu_p > 0 or gamma_d > 0:
            eps += - (nu_p**2) / (nu**2 + 1j*gamma_d*nu + 1e-30)
    for (S, nu0, gamma) in lorentz_list:
        if S != 0:
            eps += S * (nu0**2) / (nu0**2 - nu**2 - 1j*gamma*nu + 1e-30)

```



```

    return eps
def complex_n_from_eps(eps):
    N = np.sqrt(eps + 0j)
    if np.mean(np.real(N)) < 0:
        N = -N
    return N
def cos_theta_in_medium(N, theta0_rad):
    s1 = np.sin(theta0_rad) / N
    cos1 = np.sqrt(1 - s1**2)
    mask = np.real(cos1) < 0
    if np.any(mask):
        cos1[mask] = -cos1[mask]
    return cos1
def slab_reflectance_from_N(wavenumber, N, d_um, angle_deg, pfrac=0.5, sigma_nm=0.0):
    theta0 = np.deg2rad(angle_deg)
    N0 = 1.0 + 0j
    cos0 = np.cos(theta0)
    cos1 = cos_theta_in_medium(N, theta0)
    sigma_cm = sigma_nm * 1e-7
    if sigma_cm > 0:
        factor01 = np.exp(-(2*np.pi * wavenumber * sigma_cm * cos0)**2)
        factor12 = np.exp(-(2*np.pi * wavenumber * sigma_cm * np.abs(np.real(cos1)))**2)
    else:
        factor01 = 1.0
        factor12 = 1.0
    r01_s = (N0*cos0 - N*cos1) / (N0*cos0 + N*cos1) * factor01
    r12_s = (N*cos1 - N0*cos0) / (N*cos1 + N0*cos0) * factor12
    r01_p = (N*cos0 - N0*cos1) / (N*cos0 + N0*cos1) * factor01
    r12_p = (N0*cos1 - N*cos0) / (N0*cos1 + N*cos0) * factor12
    beta = 2*np.pi * wavenumber * (d_um/10000.0) * N * cos1 # d μm -> cm
    e2ib = np.exp(2j * beta)
    r_s = (r01_s + r12_s * e2ib) / (1 + r01_s * r12_s * e2ib)
    r_p = (r01_p + r12_p * e2ib) / (1 + r01_p * r12_p * e2ib)
    R_s = np.abs(r_s)**2
    R_p = np.abs(r_p)**2
    pfrac = np.clip(pfrac, 0.0, 1.0)
    R = (1 - pfrac) * R_s + pfrac * R_p
    return R
def gaussian_nodes_weights(n):
    x, w = hermgauss(n)
    return x, w / np.sqrt(np.pi)
def average_over_angle_thickness(wavenumber, N, d_um, angle_deg, pfrac, sigma_nm,
                                sigma_d_um=0.0, sigma_ang_deg=0.0, n_nodes=
5):
    if sigma_d_um <= 0 and sigma_ang_deg <= 0:
        return slab_reflectance_from_N(wavenumber, N, d_um, angle_deg, pfrac, sigma_nm)
    x, w = gaussian_nodes_weights(n_nodes)
    R_accum = np.zeros_like(wavenumber, dtype=float)
    if sigma_d_um <= 0:
        d_nodes = [d_um]; wd = np.array([1.0])
    else:
        d_nodes = d_um + np.sqrt(2)*sigma_d_um*x; wd = w

```

```

    if sigma_ang_deg <= 0:
        a_nodes = [angle_deg]; wa = np.array([1.0])
    else:
        a_nodes = angle_deg + np.sqrt(2)*sigma_ang_deg*x; wa = w
    for a, wa_i in zip(a_nodes, wa):
        for d_j, wd_j in zip(d_nodes, wd):
            Rij = slab_reflectance_from_N(wavenumber, N, d_j, a, pfrac, sigma_nm)
            R_accum += wa_i * wd_j * Rij
    return R_accum
def convolve_instrument(wavenumber, y, fwhm_cm1):
    if fwhm_cm1 <= 1e-9:
        return y.copy()
    wn = np.asarray(wavenumber)
    y = np.asarray(y)
    dw = np.median(np.diff(wn))
    if dw <= 0:
        return y.copy()
    wn_u = np.linspace(wn.min(), wn.max(), int(round((wn.max()-wn.min())/dw)) + 1)
    yi = np.interp(wn_u, wn, y)
    sigma = fwhm_cm1 / (2*np.sqrt(2*np.log(2)))
    half = int(np.ceil(4*sigma/dw))
    kx = np.arange(-half, half+1) * dw
    kernel = np.exp(-0.5*(kx/sigma)**2)
    kernel /= kernel.sum()
    yi_conv = np.convolve(yi, kernel, mode='same')
    y_back = np.interp(wn, wn_u, yi_conv)
    return y_back
# -----
# 5. 高级模型: 参数打包/计算
# -----
def model_curve_percent_advanced(wavenumber, angle_deg, params):
    d_um, eps_inf, nu_p, gamma_d, S1, nu01, g1, S2, nu02, g2, \
    pfrac, sigma_nm, sigma_d_um, sigma_ang_deg, inst_fwhm, scale, offset, slope = params
    eps = drude_lorentz_eps(wavenumber, eps_inf, (nu_p, gamma_d), [(S1, nu01, g1), (S2, nu02, g2)])
    N = complex_n_from_eps(eps)
    R = average_over_angle_thickness(
        wavenumber, N, d_um, angle_deg, pfrac, sigma_nm,
        sigma_d_um=sigma_d_um, sigma_ang_deg=sigma_ang_deg, n_nodes=5
    )
    R_pct = 100.0 * R
    R_pct_conv = convolve_instrument(wavenumber, R_pct, inst_fwhm)
    y = offset + slope*wavenumber + scale * R_pct_conv
    return y
# -----
# 6. 拟合: 单数据集 + 多起点 (实时进度)
# -----
def fit_advanced_single(wavenumber, reflectance, angle_deg, opd_initial, material_params,
                        n_restarts=6, verbose=True, tag="单数据集",
                        progress_enable=REALTIME_PROGRESS, progress_every=PROGRESS EVERY):

```

```

n_guess = material_params['n_guess']
theta = np.deg2rad(angle_deg)
d0 = opd_initial / (2*np.sqrt(max(n_guess**2 - np.sin(theta)**2, 1e-
12)))
eps_inf0 = n_guess**2
x0 = np.array([
    d0,          # d
    eps_inf0,     # eps_inf
    0.0, 200.0,   # Drude (nu_p, gamma_d)
    0.3, 1000.0, 200.0, # Lorentz1
    0.0, 2500.0, 300.0, # Lorentz2 (可为0)
    0.5, 5.0,     # pfrac, sigma_nm
    0.02, 0.20,   # sigma_d_um, sigma_ang_deg
    4.0,         # inst_fwhm
    1.0, 0.0, 0.0 # scale, offset, slope
], dtype=float)
lb = np.array([
    d0*0.90, 1.5, 0.0, 1.0,
    0.0, 200.0, 5.0,
    0.0, 500.0, 5.0,
    0.0, 0.0,
    0.0, 0.0,
    0.0,
    0.85, -15.0, -1e-3
], dtype=float)
ub = np.array([
    d0*1.10, 30.0, 4000.0, 2000.0,
    10.0, 5000.0, 2000.0,
    10.0, 8000.0, 3000.0,
    1.0, 50.0,
    0.2, 2.0,
    20.0,
    1.20, 15.0, 1e-3
], dtype=float)
def make_residuals(progressor):
    def residuals(p):
        y = model_curve_percent_advanced(wavenumber, angle_deg, p)
        res = reflectance - y
        progressor.update(res, params=p)
        return res
    return residuals
best = None
best_ssr = np.inf
total_runs = n_restarts + 1
# 用参数个数估算“真实残差调用预算”（用于进度分母）
n_params = x0.size
calls_budget = estimate_calls_budget(MAX_NFEV_LS, n_params, jac_kind='2-
point')
for k in range(total_runs):
    if k == 0:
        x_init = x0.copy()
        print(f"[{tag}] 拟合进度: 起点 1/{total_runs} (原始初值)")
    else:
        rng = np.random.default_rng(seed=1234 + k)
        span = (ub - lb)

```

```

        jitter = 0.05 * span * rng.standard_normal(x0.size)
        x_init = np.clip(x0 + jitter, lb, ub)
        x_init = 0.5*(x_init + np.clip(x0, lb, ub))
        print(f"[{tag}] 拟合进度: 起点 {k+1}/{total_runs} (轻度扰动)")
        progressor = IterPrinter(
            label=f"[{tag}] | 起点 {k + 1}/{total_runs}",
            max_nfev=calls_budget, # 用真实预算做分母
            every=progress_every,
            time_throttle=TIME_THROTTLE,
            enable=progress_enable
        )
        progressor.reset()
        res = least_squares(
            make_residuals(progres-
sor), x_init, bounds=(lb, ub), method='trf',
            jac='2-point', # 显式指定差分方式, 便于预算一致
            loss='soft_l1', f_scale=1.0,
            max_nfev=MAX_NFEV_LS, # SciPy 的内部上限
            ftol=1e-10, xtol=1e-10, gtol=1e-10, verbose=0
        )
        progressor.close_line()
        ssr = np.sum(res.fun**2)
        print(f"[{tag}] 起点 {k+1}/{total_runs} 完成: SSR = {ssr:.4f}, 函数评
估 {res.nfev} 次")
        if ssr < best_ssr:
            best_ssr = ssr
            best = res
        if verbose:
            print(f"[{tag}] 最佳 SSR = {best_ssr:.2f}, 迭代次数 {best.nfev}")
        return best.x, best_ssr
# -----
# 7. 分析与可视化 (单数据集)
# -----
def analyze_wafer_advanced(name, info, material_params):
    print(f"--- 正在处理: {name} | 材料: {material_params['name']} ---")
    wn_raw, r_raw = load_data(info['path'])
    if wn_raw is None:
        return None
    mask = (wn_raw >= info['w_range'][0]) & (wn_raw <= info['w_range'][1])
    wn, r = wn_raw[mask], r_raw[mask]
    print(f"数据筛选完成: 只分析波数在 {info['w_range']} cm-1 的数据。")
    opd0, fft_data = estimate_opd_fft(wn, r, material_params['opd_range'])
    print(f"步骤 1: FFT 初始估计 OPD = {opd0:.4f} μm")
    print("步骤 2: 改进模型拟合 (Drude+Lorentz + s/p + 角度/厚度平均 + 粗糙
度 + 仪器卷积) ...")
    p_best, ssr = fit_advanced_single(
        wn, r, info['angle'], opd0, material_params, n_restarts=6, ver-
bose=True, tag=name,
        progress_enable=REALTIME_PROGRESS, progress_every=PROGRESS EVERY
    )
    d = p_best[0]
    print("步骤 2: 优化完成。")
    print(f"厚度 d = {d:.4f} μm | p 分量占比 = {p_best[10]:.2f} | 表面粗糙
度 σ = {p_best[11]:.2f} nm")

```

```

    print(f"  $\sigma_d$  = {p_best[12]*1e3:.1f} nm |  $\sigma_{ang}$  = {p_best[13]:.2f}° | 分辨率 FWHM = {p_best[14]:.2f} cm-1")
    print(f"  $\epsilon_\infty$ ={{p_best[1]:.3f}}, Drude(vp={{p_best[2]:.1f}},  $\gamma$ ={{p_best[3]:.1f}})")
    print(f" L1(S={{p_best[4]:.3f}},  $v_0$ ={{p_best[5]:.1f}},  $\gamma$ ={{p_best[6]:.1f}});
    "
        f"L2(S={{p_best[7]:.3f}},  $v_0$ ={{p_best[8]:.1f}},  $\gamma$ ={{p_best[9]:.1f}})")
    print(f" scale={{p_best[15]:.3f}}, offset={{p_best[16]:.3f}}, slope={{p_best[17]:.2e}} (%/cm-1)")
    print(f" 最终拟合误差 (SSR) = {ssr:.2f}\n")
    print_params_18_cn(p_best, title=f"{{name}} 拟合参数 (中文)", prefix=" ")
    result = {
        'params': p_best, 'error': ssr, 'fft_data': fft_data,
        'raw_data': (wn_raw, r_raw), 'fit_range': info['w_range'],
        'angle': info['angle']}
    }
    return result
def print_params_18_cn(p, title="全部 18 个参数 (中文)", prefix=" "):
    fields = [
        ("d_um", "薄膜厚度", "μm", "lambda v: f\"{v:.4f}\""),
        ("eps_inf", "高频介电常数  $\epsilon_\infty$ ", "", "lambda v: f\"{v:.6g}\""),
        ("nu_p", "Drude 等离子频率  $\nu_p$ ", "cm-1", "lambda v: f\"{v:.1f}\""),
        ("gamma_d", "Drude 阻尼  $\gamma_d$ ", "cm-1", "lambda v: f\"{v:.1f}\""),
        ("S1", "Lorentz1 强度", "", "lambda v: f\"{v:.6g}\""),
        ("nu01", "Lorentz1 中心波数", "cm-1", "lambda v: f\"{v:.1f}\""),
        ("gamma1", "Lorentz1 宽度", "cm-1", "lambda v: f\"{v:.1f}\""),
        ("S2", "Lorentz2 强度", "", "lambda v: f\"{v:.6g}\""),
        ("nu02", "Lorentz2 中心波数", "cm-1", "lambda v: f\"{v:.1f}\""),
        ("gamma2", "Lorentz2 宽度", "cm-1", "lambda v: f\"{v:.1f}\""),
        ("pfrac", "p 偏振比例", "", "lambda v: f\"{v:.3f}\""),
        ("sigma_nm", "表面粗糙度  $\sigma$ ", "nm", "lambda v: f\"{v:.2f}\""),
        ("sigma_d_um", "厚度标准差  $\sigma_d$ ", "μm", "lambda v: f\"{v:.4f}\""),
        ("sigma_ang_deg", "入射角标准差  $\sigma_{ang}$ ", "°", "lambda v: f\"{v:.3f}\""),
        ("inst_fwhm_cm1", "仪器分辨率 FWHM", "cm-1", "lambda v: f\"{v:.2f}\""),
        ("scale", "强度缩放因子", "", "lambda v: f\"{v:.3f}\""),
        ("offset", "基线偏移", "%", "lambda v: f\"{v:.3f}\""),
    ]

```

```

        ("slope",          "基线斜率",          "%/cm^-
1", lambda v: f"{v:.2e}"),
    ]
    print(prefix + title)
    print(prefix + "-"*36)
    w1 = max(len(k) for k,_,_ in fields)
    w2 = max(len(cn) for _,cn,_ in fields)
    for i, (k, cn, unit, fmt) in enumerate(fields):
        val = float(p[i])
        print(f"{pre-
fix}[{i:02d}] {k:<{w1}} | {cn:<{w2}} = {fmt(val):>12} {unit}")
def visualize_results(results, files_to_process, title='问题三：多光束干涉模型
拟合与分析'):
    num = len(results)
    fig, axes = plt.subplots(num, 2, figsize=(16, 5*num),
                             gridspec_kw={'width_ratios': [2, 2]})
    fig.suptitle(title, fontsize=20, y=0.97)
    if num == 1:
        axes = np.array([axes])
    for i, (name, res) in enumerate(results.items()):
        ax_fit, ax_res = axes[i]
        if res is None:
            for ax in (ax_fit, ax_res):
                ax.text(0.5, 0.5, '数据处理失败', ha='center', va='center')
                ax.set_title(name)
            continue
        info = files_to_process[name]
        angle = res['angle']
        wn_raw, r_raw = res['raw_data']
        fit_range = res['fit_range']
        p = res['params']
        y_fit = model_curve_percent_advanced(wn_raw, angle, p)
        ax_fit.plot(wn_raw, r_raw, label='实验数据', lw=1.5)
        ax_fit.plot(wn_raw, y_fit, label='模型拟合曲线', color='red', ls='--
', lw=2)
        ax_fit.axvspan(fit_range[0], fit_range[1], color='green', al-
pha=0.1, label='拟合区域')
        ax_fit.set_title(f'{name} - 全谱拟合结果 (d = {p[0]:.4f} μm)')
        ax_fit.set_xlabel('波数')
        ax_fit.set_ylabel('反射率 (%)')
        ax_fit.legend()
        ax_fit.grid(True, ls=':')
        ax_fit.set_ylim(bottom=0)
        res_full = r_raw - y_fit
        ax_res.plot(wn_raw, res_full, color='green', lw=1)
        ax_res.axhline(0, color='k', ls='--', lw=1)
        ax_res.axvspan(fit_range[0], fit_range[1], color='green', alpha=0.1)
        ax_res.set_title(f'{name} - 残差分析 (SSR = {res["error"]:.2f})')
        ax_res.set_xlabel('波数')
        ax_res.set_ylabel('残差 (%)')
        ax_res.grid(True, ls=':')
        in_fit = (wn_raw >= fit_range[0]) & (wn_raw <= fit_range[1])
        res_std = np.std(res_full[in_fit]) if np.any(in_fit) else np.std(res
_full)

```

```

        if res_std > 0:
            ax_res.set_ylim(-5*res_std, 5*res_std)
        plt.tight_layout(rect=[0, 0.03, 1, 0.95])
        plt.show()
# -----
# 8. (可选) 同一晶圆多角度“联合拟合” (实时进度)
# -----
def fit_joint_advanced(datasets, material_params, n_restarts=6, verbose=True,
                       tag="联合拟合", progress_enable=REALTIME_PROGRESS, progress_every=PROGRESS_EVERY):
    wn0, r0, ang0 = datasets[0]['wn'], datasets[0]['r'], datasets[0]['angle']
    opd0, _ = estimate_opd_fft(wn0, r0, material_params['opd_range'])
    n_guess = material_params['n_guess']
    theta = np.deg2rad(ang0)
    d0 = opd0 / (2*np.sqrt(max(n_guess**2 - np.sin(theta)**2, 1e-12)))
    base0 = np.array([
        d0, n_guess**2, 0.0, 200.0,
        0.3, 1000.0, 200.0,
        0.0, 2500.0, 300.0,
        0.5, 5.0, 0.02, 0.20,
    ], dtype=float)
    base_lb = np.array([
        d0*0.90, 1.5, 0.0, 1.0,
        0.0, 200.0, 5.0,
        0.0, 500.0, 5.0,
        0.0, 0.0, 0.0, 0.0
    ], dtype=float)
    base_ub = np.array([
        d0*1.10, 30.0, 4000.0, 2000.0,
        10.0, 5000.0, 2000.0,
        10.0, 8000.0, 3000.0,
        1.0, 50.0, 0.2, 2.0
    ], dtype=float)
    m = len(datasets)
    local0 = []
    local_lb = []
    local_ub = []
    for _ in range(m):
        local0 += [4.0, 1.0, 0.0, 0.0]
        local_lb += [0.0, 0.85, -15.0, -1e-3]
        local_ub += [20.0, 1.20, 15.0, 1e-3]
    local0 = np.array(local0, dtype=float)
    local_lb = np.array(local_lb, dtype=float)
    local_ub = np.array(local_ub, dtype=float)
    x0 = np.concatenate([base0, local0])
    lb = np.concatenate([base_lb, local_lb])
    ub = np.concatenate([base_ub, local_ub])
    def unpack(x):
        base = x[:len(base0)]
        locals_flat = x[len(base0):]
        locals_list = [locals_flat[i*4:(i+1)*4] for i in range(m)]
        return base, locals_list
    def make_residuals(progressor):

```

```

def residuals(x):
    base, locals_list = unpack(x)
    d_um, eps_inf, nu_p, gamma_d, S1, nu01, g1, S2, nu02, g2, pfrac,
    sigma_nm, sigma_d, sigma_ang = base
    eps_fun = lambda wn: drude_lo-
    rentz_eps(wn, eps_inf, (nu_p, gamma_d), [(S1,nu01,g1),(S2,nu02,g2)])
    res_all = []
    for data, loc in zip(datasets, locals_list):
        wn, r, ang = data['wn'], data['r'], data['angle']
        inst_fwhm, scale, offset, slope = loc
        eps = eps_fun(wn)
        N = complex_n_from_eps(eps)
        R = average_over_angle_thick-
    ness(wn, N, d_um, ang, pfrac, sigma_nm,
                                                sigma_d_um=sigma_d, sigma_a
    ng_deg=sigma_ang, n_nodes=5)
        y = offset + slope*wn + scale * convolve_instru-
    ment(wn, 100.0*R, inst_fwhm)
        res_all.append(r - y)
        res_vec = np.concatenate(res_all)
        progressor.update(res_vec, params=np.array([d_um]))
    return res_vec
return residuals
best = None
best_ssr = np.inf
total_runs = n_restarts + 1
# 联合拟合参数个数 -> 真实残差调用预算
n_params = x0.size
calls_budget = estimate_calls_budget(MAX_NFEV_LS, n_params, jac_kind='2-
point')
for k in range(total_runs):
    if k == 0:
        x_init = x0.copy()
        print(f"[{tag}] 拟合进度: 起点 1/{total_runs} (原始初值)")
    else:
        rng = np.random.default_rng(2024+k)
        span = (ub - lb)
        jitter = 0.05 * span * rng.standard_normal(x0.size)
        x_init = np.clip(x0 + jitter, lb, ub)
        x_init = 0.5*(x_init + np.clip(x0, lb, ub))
        print(f"[{tag}] 拟合进度: 起点 {k+1}/{total_runs} (轻度扰动)")
    progressor = IterPrinter(
        label=f"[{tag}] | 起点 {k + 1}/{total_runs}",
        max_nfev=calls_budget,
        every=progress_every,
        time_throttle=TIME_THROTTLE,
        enable=progress_enable
    )
    progressor.reset()
    res = least_squares(
        make_residuals(progres-
    sor), x_init, bounds=(lb, ub), method='trf',
        jac='2-point',
        loss='soft_l1', f_scale=1.0,
        max_nfev=MAX_NFEV_LS,

```



```

        ftol=1e-10, xtol=1e-10, gtol=1e-10, verbose=0
    )
    progressor.close_line()
    ssr = np.sum(res.fun**2)
    print(f"[{tag}] 起点 {k+1}/{total_runs} 完成: SSR = {ssr:.4f}, 函数评
估 {res.nfev} 次")
    if ssr < best_ssr:
        best_ssr = ssr
        best = res
    base, locals_list = unpack(best.x)
    out = {
        'x': best.x, 'base': base, 'locals': locals_list,
        'error': best_ssr, 'nfev': best.nfev
    }
    if verbose:
        print(f"[{tag}] 最佳 SSR={best_ssr:.2f}, 迭代 {best.nfev} 次")
    return out
def main():
    PARAMS_SI = {'name': 'Si', 'n_guess': 3.42, 'n_bounds': (3.0, 4.0), 'o
pd_range': (20, 30)}
    PARAMS_SiC = {'name': 'SiC', 'n_guess': 2.55, 'n_bounds': (2.4, 2.8), '
opd_range': (35, 45)}
    files_to_process = {
        #'附件3 (Si, 10°)': {'path': './B 题/附件/附件3.xlsx', 'an-
gle': 10, 'w_range': (500, 4000), 'material': PARAMS_SI},
        #'附件4 (Si, 15°)': {'path': './B 题/附件/附件4.xlsx', 'an-
gle': 15, 'w_range': (500, 4000), 'material': PARAMS_SI},
        '附件1 (SiC, 10°)': {'path': './B 题/附件/附件1.xlsx', 'an-
gle': 10, 'w_range': (2000, 4000), 'material': PARAMS_SiC},
        '附件2 (SiC, 15°)': {'path': './B 题/附件/附件2.xlsx', 'an-
gle': 15, 'w_range': (2000, 4000), 'material': PARAMS_SiC},
    }
    results = {}
    for name, info in files_to_process.items():
        results[name] = analyze_wafer_advanced(name, info, info['material'])
    visualize_results(results, files_to_process, title='问题三: 多光束干涉模型
拟合与分析')
    print("\n--- 高级模型: 结果一致性检查 ---\n")
    if results.get('附件3 (Si, 10°)') and results.get('附件4 (Si, 15°)'):
        d10 = results['附件3 (Si, 10°)']['params'][0]
        d15 = results['附件4 (Si, 15°)']['params'][0]
        davg = np.mean([d10, d15]); diff = abs(d10-d15)/davg*100
        print(f"[Si] d(10°)={d10:.4f} μm, d(15°)={d15:.4f} μm, 平均
={davg:.4f} μm, 相对差异={diff:.3f}%")
    if results.get('附件1 (SiC, 10°)') and results.get('附件2 (SiC, 15°)'):
        d10 = results['附件1 (SiC, 10°)']['params'][0]
        d15 = results['附件2 (SiC, 15°)']['params'][0]
        davg = np.mean([d10, d15]); diff = abs(d10-d15)/davg*100
        print(f"[SiC] d(10°)={d10:.4f} μm, d(15°)={d15:.4f} μm, 平均
={davg:.4f} μm, 相对差异={diff:.3f}%")
    ENABLE_JOINT = False
    if ENABLE_JOINT:
        si_sets = []

```

```

        for key in ['附件 3 (Si, 10°)', '附件 4 (Si, 15°)']:
            wn_raw, r_raw = results[key]['raw_data']
            info = files_to_process[key]
            mask = (wn_raw >= info['w_range'][0]) & (wn_raw <= info['w_range
']] [1])
            si_sets.append({'wn': wn_raw[mask], 'r': r_raw[mask], 'an-
gle': info['angle']})
            si_joint = fit_joint_advanced(si_sets, PARAMS_SI, n_restarts=6, ver-
bose=True, tag="Si 联合拟合",
                                     progress_enable=REALTIME_PRO-
GRESS, progress_every=PROGRESS_EVERY)
            print(f"Si 联合拟合完成: 共
享 d={si_joint['base'][0]:.4f} μm, SSR={si_joint['error']:.2f}")
            sic_sets = []
            for key in ['附件 1 (SiC, 10°)', '附件 2 (SiC, 15°)']:
                wn_raw, r_raw = results[key]['raw_data']
                info = files_to_process[key]
                mask = (wn_raw >= info['w_range'][0]) & (wn_raw <= info['w_range
']] [1])
                sic_sets.append({'wn': wn_raw[mask], 'r': r_raw[mask], 'an-
gle': info['angle']})
                sic_joint = fit_joint_advanced(sic_sets, PARAMS_SIC, n_re-
starts=6, verbose=True, tag="SiC 联合拟合",
                                     progress_enable=REALTIME_PRO-
GRESS, progress_every=PROGRESS_EVERY)
                print(f"SiC 联合拟合完成: 共
享 d={sic_joint['base'][0]:.4f} μm, SSR={sic_joint['error']:.2f}")
            if __name__ == '__main__':
                main()

```

附录 C-5

贝叶斯拓展

```

import time
import emcee
import arviz as az
from scipy.special import gammaln
import warnings
import numpy as np
import pandas as pd
from q3_nihe import *
# =====
# 10. Bayesian 扩展 (基于 emcee 的全参数后验推断)
# =====
# ----- 10.1 实用工具 -----
def student_t_logpdf(residuals, sigma, nu=4.0):
    """Student-t 对数似然 (独立同分布), 返回 sum(log p) 与逐点 log p 向量"""
    if sigma <= 0 or nu <= 1e-6:
        return -np.inf, None
    r = np.asarray(residuals, dtype=float)
    c = gammaln((nu+1)/2) - gam-
maln(nu/2) - 0.5*np.log(nu*np.pi) - np.log(sigma)

```

```

ll_i = c - ((nu+1)/2) * np.log(1 + (r/sigma)**2 / nu)
return np.sum(ll_i), ll_i
def normal_logpdf(x, mu, sig):
    if sig <= 0:
        return -np.inf
    return -0.5*np.log(2*np.pi*sig*sig) - 0.5*((x-mu)/sig)**2
def halfnormal_logpdf(x, sig):
    if x < 0 or sig <= 0:
        return -np.inf
    return np.log(2.0) - 0.5*np.log(2*np.pi*sig*sig) - 0.5*(x/sig)**2
def halfcauchy_logpdf(x, scale):
    if x <= 0 or scale <= 0:
        return -np.inf
    return np.log(2/np.pi) - np.log(scale*(1+(x/scale)**2))
def within_bounds(x, lb, ub):
    return np.all(x >= lb) and np.all(x <= ub)
def jitter_within_bounds(x, lb, ub, rel_scale=0.02, rng=None):
    rng = np.random.default_rng() if rng is None else rng
    span = (ub - lb)
    z = x + rel_scale * span * rng.standard_normal(x.size)
    return np.clip(z, lb, ub)
def aic_bic_from_ssr(ssr, n, k):
    # 高斯似然假设下的 AIC/BIC (供对比)
    # AIC = 2k + n*ln(SSR/n) + 常数; BIC = k*ln(n) + n*ln(SSR/n) + 常数
    # 常数项在比较同一数据集不同模型时抵消
    return 2*k + n*np.log(max(ssr/n, 1e-
30)), k*np.log(n) + n*np.log(max(ssr/n, 1e-30))
def waic_from_loglik_matrix(loglik_matrix):
    """
    输入: S x N 的逐点 log-lik 样本矩阵 (S=样本数, N=数据点数)
    返回: WAIC、lppd、p_waic、se
    """
    loglik = np.asarray(loglik_matrix)
    # lppd = sum Log(mean exp(loglik_s,i))
    # 为数值稳定, 减去每列最大值再加回
    m = np.max(loglik, axis=0, keepdims=True)
    lppd = np.sum(np.log(np.mean(np.exp(loglik - m), axis=0)) + m.ravel())
    p_waic = np.sum(np.var(loglik, axis=0, ddof=1))
    waic = -2*(lppd - p_waic)
    # 标准误差 (近似)
    waic_i = -2*(np.log(np.mean(np.exp(log-
lik - m), axis=0)) + m.ravel() - np.var(loglik, axis=0, ddof=1))
    se = np.sqrt(np.var(waic_i, ddof=1) * loglik.shape[1])
    return waic, lppd, p_waic, se
def numerical_grad_wrt_d(wn, angle, p18, eps=1e-4):
    """对厚度 d 的数值梯度  $\partial y / \partial d$  (用于 Fisher/CRLB), 中心差分"""
    p_hi = p18.copy(); p_lo = p18.copy()
    p_hi[0] += eps; p_lo[0] -= eps
    y_hi = model_curve_percent_advanced(wn, angle, p_hi)
    y_lo = model_curve_percent_advanced(wn, angle, p_lo)
    return (y_hi - y_lo) / (2*eps)
def crlb_for_thickness(wn, angle_deg, p18, sigma_obs):
    """单数据集对 d 的 CRLB:  $\text{Var}(d) \geq \sigma^2 / \sum (\partial y / \partial d)^2$ """
    dyd = numerical_grad_wrt_d(wn, angle_deg, p18)
    fisher = np.sum((dyd / sigma_obs)**2)

```

```

    if fisher <= 1e-30:
        return np.inf
    return 1.0 / fisher
# ----- 10.2 单数据集: 先验、似然、后验 -----
class BayesSingle:
    """
    使用已有 18 参数模型 + 1 个观测噪声参数 (log_sigma), 做 Bayesian 推断
    """
    def __init__(self, wavenumber, reflectance, angle_deg, material_params,
                  lsq_best=None, opd_initial=None, nu_t=4.0):
        self.wn = np.asarray(wavenumber)
        self.r = np.asarray(reflectance)
        self.angle = angle_deg
        self.mat = material_params
        self.nu_t = float(nu_t) # Student-t 自由度, 默认 4 (抗异常值)
        self.x0, self.lb, self.ub = self._build_initial_and_bounds(opd_ini-
tial)
        # 如果给了最小二乘的最优点, 用它覆盖初值
        if lsq_best is not None:
            if lsq_best.shape[0] == 18:
                self.x0 = np.clip(lsq_best.copy(), self.lb, self.ub)
        # 估计观测噪声初值
        y0 = model_curve_percent_advanced(self.wn, self.angle, self.x0)
        resid = self.r - y0
        sigma0 = max(np.std(resid), 1e-3) # 单位: 百分比
        self.log_sigma0 = np.log(sigma0)
        # Walker 初始化范围
        self.walker_rel_jitter = 0.02
    def _build_initial_and_bounds(self, opd_initial):
        # 复制 fit_advanced_single 的初始化和边界
        n_guess = self.mat['n_guess']
        if opd_initial is None:
            opd_initial, _ = esti-
mate_opd_fft(self.wn, self.r, self.mat['opd_range'])
            theta = np.deg2rad(self.angle)
            d0 = opd_initial / (2*np.sqrt(max(n_guess**2 - np.sin(theta)**2, 1e-
12)))
            eps_inf0 = n_guess**2
            x0 = np.array([
                d0, eps_inf0, 0.0, 200.0,
                0.3, 1000.0, 200.0,
                0.0, 2500.0, 300.0,
                0.5, 5.0,
                0.02, 0.20,
                4.0,
                1.0, 0.0, 0.0
            ], dtype=float)
            lb = np.array([
                d0*0.90,
                1.5,
                0.0, 1.0,
                0.0, 200.0, 5.0,
                0.0, 500.0, 5.0,
                0.0, 0.0,
                0.0, 0.0,

```

```

        0.0,
        0.85, -15.0, -1e-3
    ], dtype=float)
    ub = np.array([
        d0*1.10,
        30.0,
        4000.0, 2000.0,
        10.0, 5000.0, 2000.0,
        10.0, 8000.0, 3000.0,
        1.0, 50.0,
        0.2, 2.0,
        20.0,
        1.20, 15.0, 1e-3
    ], dtype=float)
    return x0, lb, ub
def log_prior(self, z):
    """
    z = [p18..., log_sigma]: 在边界内用“弱信息先验”，边界外返回 -inf
    先验设置策略：
    - 对 18 个物理/仪器参数：在 [lb,ub] 内用近似平坦先验（Uniform），
      并对 S1/S2 与  $\gamma_1/\gamma_2$  添加轻微收缩（半正态）避免非物理过大
    - 噪声 sigma: HalfCauchy(scale=5) on  $\sigma$ （以 % 为单位）
    """
    p = np.array(z[:-1], dtype=float)
    log_sig = float(z[-1])
    sig = np.exp(log_sig)
    if not within_bounds(p, self.lb, self.ub):
        return -np.inf
    # 弱收缩项（可按需调整强度）
    S1, S2 = p[4], p[7]
    g1, g2 = p[6], p[9]
    lp = 0.0
    # 对振子强度与宽度做轻微半正态收缩（scale 越大越弱）
    lp += halfnormal_logpdf(S1, sig=5.0) if S1 >= 0 else -np.inf
    lp += halfnormal_logpdf(S2, sig=5.0) if S2 >= 0 else -np.inf
    lp += halfnormal_logpdf(g1, sig=1000.0) if g1 >= 0 else -np.inf
    lp += halfnormal_logpdf(g2, sig=1000.0) if g2 >= 0 else -np.inf
    # 噪声先验: HalfCauchy(5)
    lp += halfcauchy_logpdf(sig, scale=5.0) + log_sig # 变换加
    上 Log|dσ/dLogσ| = Logσ
    return lp
def log_likelihood(self, z, return_pointwise=False):
    p = np.array(z[:-1], dtype=float)
    sig = float(np.exp(z[-1]))
    y = model_curve_percent_advanced(self.wn, self.angle, p)
    resid = self.r - y
    ll_sum, ll_i = student_t_logpdf(resid, sigma=sig, nu=self.nu_t)
    if return_pointwise:
        return ll_sum, ll_i
    return ll_sum
def log_posterior(self, z):
    lp = self.log_prior(z)
    if not np.isfinite(lp):
        return -np.inf
    ll = self.log_likelihood(z)

```

```

        return lp + ll
    def sample(self, nwalk-
ers=64, nsteps=4000, burnin=1500, thin=1, seed=2025, progress=True):
        """
        运行 emcee 采样; 返回 {samples, log_prob, sampler, info}
        """
        rng = np.random.default_rng(seed)
        p18 = self.x0.copy()
        # 初始 walker: 围绕最优点做小扰动
        init = []
        for _ in range(nwalkers):
            p0 = jit-
            ter_within_bounds(p18, self.lb, self.ub, rel_scale=self.walker_rel_jit-
            ter, rng=rng)
            ls0 = self.log_sigma0 + 0.05*rng.standard_normal()
            init.append(np.concatenate([p0, [ls0]]))
        init = np.array(init)
        ndim = 19 # 18 + log_sigma
        sampler = emcee.EnsembleSampler(nwalkers, ndim, self.log_posterior)
        t0 = time.time()
        state = sampler.run_mcmc(init, nsteps, progress=progress)
        t1 = time.time()
        # 丢弃 burn-in, thin
        samples = sampler.get_chain(discard=burnin, thin=thin, flat=True)
        log_prob = sampler.get_log_prob(dis-
        card=burnin, thin=thin, flat=True)
        info = {
            'runtime_s': t1 - t0,
            'accept_frac': np.mean(sampler.acceptance_fraction),
            'n_effective': samples.shape[0]
        }
        self._last_sampler = sampler
        self._last_state = state
        self._last_samples = samples
        self._last_logprob = log_prob
        return {'samples': samples, 'log_prob': log_prob, 'sampler': sam-
        pler, 'info': info}
    def summarize(self, samples, param_names=None, credible=0.95):
        """
        返回每个参数的 中位数、低/高可信区间
        """
        if param_names is None:
            param_names = [
                'd_um', 'eps_inf', 'nu_p', 'gamma_d',
                'S1', 'nu01', 'gamma1', 'S2', 'nu02', 'gamma2',
                'pfrac', 'sigma_nm', 'sigma_d_um', 'sigma_ang_deg',
                'inst_fwhm_cm1', 'scale', 'offset', 'slope', 'log_sigma'
            ]
        qlo = (1-credible)/2
        qhi = 1 - qlo
        qs = np.quantile(samples, [qlo, 0.5, qhi], axis=0).T
        summary = pd.DataFrame(qs, col-
        umns=[f'{int(100*qlo)}%', '50%', f'{int(100*qhi)}%'], index=param_names)
        return summary
    def posterior_predictive(self, samples, ns=200, seed=0):

```

```

"""
后验预测: 返回 {wn, y_mean, y_lo, y_hi} (带 Student-t 噪声)
"""
rng = np.random.default_rng(seed)
idx = rng.choice(samples.shape[0], size=min(ns, samples.shape[0]), replace=False)
Y = []
for j in idx:
    z = samples[j]
    p = z[:-1]; sig = float(np.exp(z[-1]))
    y = model_curve_percent_advanced(self.wn, self.angle, p)
    # 采样学生 t 噪声
    # 用标准 t -> 缩放到 sigma
    noise = rng.standard_t(df=self.nu_t, size=self.wn.size) * sig * np.sqrt((self.nu_t - 2)/self.nu_t) \
        if self.nu_t > 2 else rng.standard_t(df=self.nu_t, size=self.wn.size) * sig
    Y.append(y + noise)
Y = np.array(Y)
y_mean = Y.mean(axis=0)
y_lo, y_hi = np.quantile(Y, [0.05, 0.95], axis=0)
return {'wn': self.wn, 'y_mean': y_mean, 'y_lo': y_lo, 'y_hi': y_hi}
def compute_waic(self, samples, ns=300, seed=0):
"""
从后验子样本估计 WAIC: 返回 (waic, se, lppd, p_waic)
"""
rng = np.random.default_rng(seed)
idx = rng.choice(samples.shape[0], size=min(ns, samples.shape[0]), replace=False)
loglik_mat = []
for j in idx:
    z = samples[j]
    ll_sum, ll_i = self.log_likelihood(z, return_pointwise=True)
    loglik_mat.append(ll_i)
loglik_mat = np.array(loglik_mat) # S x N
waic, lppd, pwaic, se = waic_from_loglik_matrix(loglik_mat)
return waic, se, lppd, pwaic
def quick_crlb(self, posterior_samples=None):
"""
使用后验均值的参数与后验均值的噪声估计 CRLB(d) (可作为理论下限)
"""
if posterior_samples is None:
    posterior_samples = self._last_samples
zbar = np.mean(posterior_samples, axis=0)
pbar = zbar[:-1]; sig = float(np.exp(zbar[-1]))
var_d = crlb_for_thickness(self.wn, self.angle, pbar, sigma_obs=sig)
return var_d
def main():
    # 材料初值
    PARAMS_SI = {'name': 'Si', 'n_guess': 3.42, 'n_bounds': (3.0, 4.0), 'opd_range': (20, 30)}
    PARAMS_SiC = {'name': 'SiC', 'n_guess': 2.55, 'n_bounds': (2.4, 2.8), 'opd_range': (35, 45)}
    files_to_process = {

```

```

    '附件 3 (Si, 10°)': {'path': './附件/附件 3.xlsx', 'angle': 10, 'w_range': (400, 4000), 'material': PARAMS_SI},
    '附件 4 (Si, 15°)': {'path': './附件/附件 4.xlsx', 'angle': 15, 'w_range': (400, 4000), 'material': PARAMS_SI},
    '附件 1 (SiC, 10°)': {'path': './附件/附件 1.xlsx', 'angle': 10, 'w_range': (2000, 4000), 'material': PARAMS_SIC},
    '附件 2 (SiC, 15°)': {'path': './附件/附件 2.xlsx', 'angle': 15, 'w_range': (2000, 4000), 'material': PARAMS_SIC}
}
# 单数据集拟合
results = {}
for name, info in files_to_process.items():
    results[name] = analyze_wafer_advanced(name, info, info['material'])
    res = results[name]
    wn_raw, r_raw = res['raw_data']; angle = res['angle']
    fit_range = res['fit_range']
    mask = (wn_raw >= fit_range[0]) & (wn_raw <= fit_range[1])
    wn, r = wn_raw[mask], r_raw[mask]
    # 初值与最优点 (用 analyze_wafer_advanced 返回的参数)
    p_best = res['params'] # 18 维点估计
    # 构建并采样
    bayes = BayesSingle(wn, r, angle, material_params=files_to_process[name]['material'],
                        lsq_best=p_best, opd_initial=None, nu_t=4.0)
    out = bayes.sample(nwalkers=64, nsteps=4000, burnin=1500, thin=2, seed=2025, progress=True)
    samples = out['samples']; print(out['info'])
    # 汇总与可视化
    summary = bayes.summarize(samples)
    print("\n[单数据集] 参数后验汇总 (中位数与 95%区间): ")
    print(summary)
    waic, se, lppd, pwaic = bayes.compute_waic(samples, ns=300)
    print(f"WAIC={waic:.1f} ± {se:.1f} (lppd={lppd:.1f}, p_waic={pwaic:.1f})")
    # 后验预测检查 (PPC)
    ppc = bayes.posterior_predictive(samples, ns=200)
    plt.figure(figsize=(10,4))
    plt.fill_between(ppc['wn'], ppc['y_lo'], ppc['y_hi'], color='C0', alpha=0.25, label='后验预测 90% 区间')
    plt.plot(wn, r, 'k-', lw=1, label='观测')
    plt.legend(); plt.xlabel('波数 (cm-1)'); plt.ylabel('反射率 (%)'); plt.grid(ls=':')
    # 厚度的可信区间
    d_samples = samples[:,0]
    print(f"d (μm): 中位数={np.median(d_samples):.4f}, 95%CI=({np.quantile(d_samples,0.025):.4f}, {np.quantile(d_samples,0.975):.4f})")
    var_d = bayes.quick_crlb(samples)
    print(f"CRLB(d): 标准差下限 ≈ {np.sqrt(var_d):.4f} μm")
if __name__ == '__main__':
    main()

```